ICPC Contest Handbook

Data structures and algorithms explained for contests:
idea, classic problem, approach and tested C++ code

Basics · Math · Binary Search & Greedy · Dynamic Programming · Data Structures
Strings · Graphs · Trees · Game Theory · Geometry · Pattern Index · Full Templates

Every full program in this book was compiled and checked against its sample.

Contents

Chapter 1. Contest Basics 5
- 1.1 The contest template 5
- 1.2 How to read the constraints 5
- 1.3 Mistakes that cost the most penalties 6
- 1.4 How to attack a problem 6
- 1.5 STL you must know 7

Chapter 2. Number Theory and Math 8
- 2.1 Modular arithmetic 8
- 2.2 Binary exponentiation 8
- 2.3 GCD, LCM and extended Euclid 9
- 2.4 Sieve of Eratosthenes and smallest prime factor 9
- 2.5 Divisors: count, sum, list 10
- 2.6 Binomial coefficients nCr mod p 11
- 2.7 Euler's totient φ(n) 12
- 2.8 Inclusion–exclusion 12
- 2.9 Matrix exponentiation 13

Chapter 3. Prefix Sums, Binary Search, Two Pointers, Greedy 15
- 3.1 Prefix sums 15
- 3.2 Binary search 16
- 3.3 Two pointers and sliding window 17
- 3.4 Greedy 18
- 3.5 Sorting tricks and sweep line 19
- 3.6 Meet in the middle 20

Chapter 4. Dynamic Programming 21
- 4.1 How to think about DP 21
- 4.2 Counting ways: Dice Combinations 21
- 4.3 Coin problems (the three classic versions) 21
- 4.4 Grid DP 22
- 4.5 0/1 Knapsack 23
- 4.6 Longest Increasing Subsequence (LIS) in O(n log n) 24
- 4.7 LCS and Edit Distance (two strings) 24
- 4.8 Interval DP 25
- 4.9 Bitmask DP 25
- 4.10 Digit DP 26
- 4.11 DP on trees 27
- 4.12 DP on a DAG 28
- 4.13 DP optimization notes 29

Chapter 5. Data Structures 30
- 5.1 Monotonic stack 30
- 5.2 Sliding window maximum (monotonic deque) 30
- 5.3 Fenwick tree (Binary Indexed Tree) 31
- 5.4 Segment tree 31
- 5.5 Lazy segment tree (range update + range query) 32
- 5.6 Sparse table (static range min/max in O(1)) 34
- 5.7 Ordered set (policy-based data structure) 34
- 5.8 DSU (Union-Find) in practice 35

5.9 Square root decomposition and Mo's algorithm 35

Chapter 6. Strings 36

6.1 String basics in C++ 36
6.2 Prefix function and KMP 36
6.3 Z-function 37
6.4 String hashing 37
6.5 Trie 39
6.6 Palindromes and Manacher 40
6.7 Quick reference: which string tool? 41

Chapter 7. Graphs 42

7.1 Representing a graph 42
7.2 BFS: shortest path in an unweighted graph 42
7.3 DFS / connected components 43
7.4 Bipartite check (2-coloring) 44
7.5 Dijkstra 44
7.6 Floyd-Warshall 45
7.7 Bellman-Ford: negative cycles 46
7.8 Topological sort and cycle detection 46
7.9 Minimum spanning tree and DSU 47
7.10 Strongly connected components 48
7.11 Max flow 49
7.12 Graph modelling tips 50

Chapter 8. Trees 51

8.1 Subtree sizes 51
8.2 Tree diameter 51
8.3 Binary lifting: k-th ancestor and LCA 52
8.4 Euler tour: subtree queries become range queries 53
8.5 Rerooting technique 54
8.6 Tree problem checklist 55

Chapter 9. Game Theory 56

9.1 Winning and losing positions 56
9.2 Nim 56
9.3 Sprague-Grundy theorem 57
9.4 Games on graphs 59
9.5 Score games (minimax) 59
9.6 Game theory checklist 60

Chapter 10. Geometry, Bit Tricks and Other Topics 61

10.1 Geometry basics with integers 61
10.2 Convex hull (Andrew's monotone chain) 62
10.3 Bit manipulation tricks 63
10.4 Constructive problems 64
10.5 Interactive problems 64
10.6 Probability and expected value 65

Chapter 11. Problem Pattern Index (read this during the contest) 66

11.1 From keywords to techniques 67
11.2 Standard tricks worth remembering 68
11.3 Kadane's algorithm (maximum subarray sum) 68
11.4 Final contest-day checklist 69

Appendix A. Complete Graph Template (24 algorithms) 70
A.0 Header and table of contents 70
A.1 BFS — shortest path in an UNWEIGHTED graph 71
A.2 GRID BFS — shortest path on a 2D grid ('#' = wall, '.' = free) 72
A.3 DFS — iterative (recursive DFS can overflow the stack for n ~ 1e6) 72
A.4 0-1 BFS — shortest path when every weight is 0 or 1 73
A.5 DSU (Disjoint Set Union / Union-Find) 74
A.6 TOPOLOGICAL SORT (Kahn's algorithm) — DIRECTED graph 74
A.7 BIPARTITE CHECK — can we color nodes with 2 colors so every edge 75
A.8 DIJKSTRA — single-source shortest path, NON-NEGATIVE weights 76
A.9 BELLMAN-FORD — shortest paths with NEGATIVE weights 76
A.10 FLOYD-WARSHALL — all-pairs shortest paths (n <= ~500) 77
A.11 KRUSKAL — Minimum Spanning Tree 78
A.12 PRIM — Minimum Spanning Tree (good for dense graphs / adjacency lists) 78
A.13 LCA — Lowest Common Ancestor with BINARY LIFTING (on a TREE) 79
A.14 BRIDGES & ARTICULATION POINTS — UNDIRECTED graph 80
A.15 SCC — Strongly Connected Components (Kosaraju) — DIRECTED graph 81
A.16 2-SAT — boolean variables x0..x(n-1), clauses of form (A OR B) 82
A.17 MAX FLOW — DINIC 83
A.18 MIN COST MAX FLOW — SPFA (Bellman-Ford with queue) 85
A.19 BIPARTITE MATCHING — HOPCROFT-KARP 86
A.20 HUNGARIAN ALGORITHM — minimum cost assignment 87
A.21 EULER PATH / CIRCUIT — HIERHOLZER (uses every EDGE exactly once) 88
A.22 TREE DIAMETER — longest path in a tree 89
A.23 HEAVY-LIGHT DECOMPOSITION — path queries/updates on a tree 90
A.24 CENTROID DECOMPOSITION (skeleton) 91
A.25 Demo main (shows how to call every algorithm) 92
Appendix B. Bitmask DP Template (Elevator Rides worked solution) 96

Chapter 1. Contest Basics

1.1 The contest template

Start every solution from the same file. It saves time and avoids silly mistakes.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int,int>;
using pll = pair<ll,ll>;
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
const ll MOD = 1e9 + 7;
const ll INF = 1e18;

void solve() {
    // read input for ONE test case and print its answer
}

int main() {
    ios::sync_with_stdio(false);   // fast input/output
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;                    // uncomment when there are many test cases
    while (t--) solve();
    return 0;
}
```

Notes:

- `ios::sync_with_stdio(false); cin.tie(nullptr);` makes `cin`/`cout` as fast as `scanf`/`printf`. Do not mix `cin` with `scanf` after it.
- Print `"\n"`, not `endl`. `endl` flushes the output every time and can cause Time Limit Exceeded.
- Interactive problems are the exception: there you MUST flush (`cout << x << endl;`).

1.2 How to read the constraints

The constraint on n tells you which complexity the intended solution has. A judge does about **10^8 simple operations per second**.

n up to	Allowed complexity	Typical idea
10	O(n!) , O(n^6)	Try all permutations
20	O(2^n · n)	Bitmask DP, subsets
40	O(2^(n/2))	Meet in the middle
500	O(n^3)	Floyd-Warshall, interval DP
5000	O(n^2)	2D DP, all pairs
10^5 – 10^6	O(n log n)	Sorting, binary search, segment tree, Dijkstra
10^7	O(n)	Prefix sums, two pointers, sieve
10^9 – 10^18	O(log n) or O(sqrt n)	Math formula, binary exponent, binary search

1.3 Mistakes that cost the most penalties

1. **Integer overflow.** If a value can exceed about 2·10^9, use `long long`. Products like `a * b` of two ints overflow before being stored, so write `1LL * a * b`.
2. **Array size.** Use `n + 5` or vectors. Out-of-bounds access gives random wrong answers.
3. **Multiple test cases.** Reset global arrays between test cases. Do not use `memset` on a big array for each of 10^5 tests; clear only the part you used.
4. **Uninitialized variables.** Local variables start with garbage; always initialize.
5. **Wrong output format.** Check "YES" vs "Yes", spaces, and new lines.
6. **Recursion depth.** Recursive DFS on 10^6 nodes can crash with stack overflow. Use an iterative version.
7. **Floating point.** Compare doubles with an epsilon: `fabs(a - b) < 1e-9`. Prefer integers when possible.
8. **Modulo with negatives.** In C++, `-7 % 3 == -1`. Use `((x % MOD) + MOD) % MOD`.

1.4 How to attack a problem

1. Read the statement twice. Write down the input limits.
2. Work out the sample by hand. Many ideas appear here.
3. Ask: what is the brute force? What is its complexity? What does the constraint allow?
4. Look for a known pattern: sorting + greedy, binary search on the answer, prefix sums, DP, graph.
5. Before coding, check the idea on the samples and on a tiny edge case (n = 1, all equal, all zero).
6. If you get Wrong Answer, write a brute force and compare outputs on random small tests (stress testing).

```
// Stress test idea (run in a terminal loop):
//   gen.cpp  -> prints a random small test
//   good.cpp -> brute force, surely correct
//   fast.cpp -> your solution
```

```
// for i in $(seq 1 1000); do ./gen > in; ./good < in > a; ./fast < in > b;
//   if ! cmp -s a b; then echo DIFF; cat in; break; fi; done
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
int rnd(int l, int r) { return uniform_int_distribution<int>(l, r)(rng); }
```

1.5 STL you must know

Container	Use	Key operations
`vector<int>`	Dynamic array	`push_back`, `sort`, `lower_bound`
`pair`, `tuple`	Group values; sorts by first, then second	`auto [a, b] = p;`
`set<int>`	Sorted unique values	`insert`, `erase`, `find`, `lower_bound` in O(log n)
`multiset<int>`	Sorted values with duplicates	erase ONE copy: `ms.erase(ms.find(x))`
`map<K,V>`	Sorted key → value	`mp[k]++`, iterate in key order
`unordered_map`	Hash map, O(1) average	Can be hacked; use `map` if unsure
`priority_queue<int>`	Max-heap	Min-heap: `priority_queue<int, vector<int>, greater<int>>`
`deque<int>`	Push/pop at both ends	Sliding window maximum
`stack`, `queue`	LIFO / FIFO	DFS / BFS
`bitset<N>`	Fast boolean array	`count`, shifts, 64x faster subset-sum DP

```
vector<int> a = {5, 1, 4, 1, 3};
sort(a.begin(), a.end());                       // 1 1 3 4 5
int pos = lower_bound(all(a), 3) - a.begin();   // first index with a[i] >= 3  -
> 2
int cnt = upper_bound(all(a), 1) - lower_bound(all(a), 1); // how many 1s -> 2
a.erase(unique(all(a)), a.end());               // remove duplicates (after
sort): 1 3 4 5
reverse(all(a));
int mx = *max_element(all(a));
ll sum = accumulate(all(a), 0LL);              // 0LL so the sum is long long
sort(all(a), greater<int>());                   // descending
next_permutation(all(a));                       // next arrangement (for brute
force)
```

Chapter 2. Number Theory and Math

2.1 Modular arithmetic

Answers are often huge, so problems ask for the answer **modulo** M (usually 10^9+7 or 998244353, both prime).

Rules you can use freely:

- (a + b) mod M = ((a mod M) + (b mod M)) mod M
- (a · b) mod M = ((a mod M) · (b mod M)) mod M — multiply in `long long`!
- (a − b) mod M = ((a − b) mod M + M) mod M — add M so it is never negative
- Division is NOT allowed directly. Divide by b means multiply by the **modular inverse** of b.

If M is prime, the inverse of b is b^(M−2) mod M (Fermat's little theorem).

2.2 Binary exponentiation

What it does: computes a^b mod M in O(log b) instead of O(b).

Idea: a^13 = a^8 · a^4 · a^1 because 13 = 1101 in binary. Square a repeatedly; multiply it into the answer when the current bit of b is 1.

Problem (CSES 1095 Exponentiation): answer n queries; each gives a and b, print a^b mod 10^9+7. (n ≤ 2·10^5, a, b ≤ 10^9.)

Approach: one `binpow` call per query. Note 0^0 = 1 by convention here.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1e9 + 7;

ll binpow(ll a, ll b, ll m = MOD) {
    a %= m;
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % m;   // current bit is 1 -> use this power
        a = a * a % m;                   // a, a^2, a^4, a^8, ...
        b >>= 1;
    }
    return res;
}
ll modinv(ll a, ll m = MOD) { return binpow(a, m - 2, m); } // only when m is
prime

int main() {
    int n; cin >> n;
    while (n--) {
        ll a, b; cin >> a >> b;
        cout << binpow(a, b) << "\n";
```

```
    }
}
```

Trick: a^(b^c) mod p (p prime): by Fermat, reduce the exponent mod (p−1): `binpow(a, binpow(b, c, p-1), p)` (CSES 1712).

2.3 GCD, LCM and extended Euclid

- gcd(a, b) = gcd(b, a mod b), gcd(a, 0) = a. Use `__gcd(a, b)` or `std::gcd` (C++17).
- lcm(a, b) = a / gcd(a, b) · b — divide FIRST to avoid overflow.
- **Extended Euclid** finds x, y with a·x + b·y = gcd(a, b). Used for the inverse when M is not prime, and for equations a·x + b·y = c (solvable only if gcd divides c).

```
// returns g = gcd(a,b) and sets x, y so that a*x + b*y = g
ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
// inverse of a mod m when gcd(a, m) = 1 (m need not be prime)
ll invGeneral(ll a, ll m) {
    ll x, y;
    extgcd(a, m, x, y);
    return (x % m + m) % m;
}
```

2.4 Sieve of Eratosthenes and smallest prime factor

What it does: finds all primes up to N in O(N log log N). Storing the **smallest prime factor** (spf) of each number lets you factorize any x ≤ N in O(log x).

Problem: given q numbers x ≤ 10^6, print the prime factorization of each.

Approach: build spf once, then repeatedly divide x by spf[x].

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1000000;
int spf[N + 1];                 // spf[x] = smallest prime dividing x
vector<int> primes;

void sieve() {
    for (int i = 2; i <= N; i++) {
        if (spf[i] == 0) {      // nobody marked i -> i is prime
            spf[i] = i;
            primes.push_back(i);
            for (long long j = 1LL * i * i; j <= N; j += i)
                if (spf[j] == 0) spf[j] = i;
        }
    }
```

```
}

int main() {
    sieve();
    int q; cin >> q;
    while (q--) {
        int x; cin >> x;
        bool first = true;
        while (x > 1) {
            int p = spf[x], e = 0;
            while (x % p == 0) { x /= p; e++; }
            if (!first) cout << ' ';
            cout << p << '^' << e;
            first = false;
        }
        cout << "\n";
    }
}
```

Factorize a single big x (up to 10^12): trial division up to sqrt(x) in O(sqrt x).

```
vector<pair<long long,int>> factorize(long long x) {
    vector<pair<long long,int>> f;
    for (long long p = 2; p * p <= x; p++) {
        if (x % p) continue;
        int e = 0;
        while (x % p == 0) { x /= p; e++; }
        f.push_back({p, e});
    }
    if (x > 1) f.push_back({x, 1});     // what is left is a prime
    return f;
}
```

2.5 Divisors: count, sum, list

If x = p1^e1 · p2^e2 · ... then

- number of divisors = (e1+1)(e2+1)...
- sum of divisors = Π (p^(e+1) − 1) / (p − 1)

Problem (CSES 1713 Counting Divisors): n ≤ 10^5 queries, x ≤ 10^6, print the number of divisors of x.

Approach: a divisor sieve: for every d, add 1 to all multiples of d. Total work N/1 + N/2 + ... = O(N log N).

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1000000;
int cntDiv[N + 1];

int main() {
    for (int d = 1; d <= N; d++)
        for (int m = d; m <= N; m += d) cntDiv[m]++;   // d divides m
```

```
    int n; cin >> n;
    while (n--) { int x; cin >> x; cout << cntDiv[x] << "\n"; }
}
```

List divisors of one x in O(sqrt x): loop i from 1 while i·i ≤ x; if i divides x, add i and x / i (once when equal).

2.6 Binomial coefficients nCr mod p

nCr = n! / (r! · (n−r)!). Precompute factorials and inverse factorials once, then each query is O(1).

Problem (CSES 1079 Binomial Coefficients): n ≤ 10^5 queries; each gives a, b ≤ 10^6; print aCb mod 10^9+7.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1e9 + 7;
const int N = 1000000;
ll fact[N + 1], inv_fact[N + 1];

ll binpow(ll a, ll b) {
    ll r = 1; a %= MOD;
    while (b) { if (b & 1) r = r * a % MOD; a = a * a % MOD; b >>= 1; }
    return r;
}
void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    inv_fact[N] = binpow(fact[N], MOD - 2);              // one inverse
    for (int i = N; i > 0; i--) inv_fact[i - 1] = inv_fact[i] * i % MOD; // the
rest in O(N)
}
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD;
}

int main() {
    precompute();
    int q; cin >> q;
    while (q--) { int a, b; cin >> a >> b; cout << nCr(a, b) << "\n"; }
}
```

Counting formulas worth remembering

Question	Answer
Ways to choose r of n items	nCr
Paths in a grid from (0,0) to (n,m), moving right/down	(n+m) C n
Ways to split n identical balls into k boxes (boxes may be empty)	(n+k−1) C (k−1) — stars and bars
Balanced bracket sequences of length 2n	Catalan: (2n C n) / (n+1)
Permutations with no fixed point (derangements)	D(n) = (n−1)(D(n−1) + D(n−2)), D(1)=0, D(2)=1
Arrangements of a multiset	n! / (c1! c2! ...)

For small n (≤ 5000) without a prime modulus, use Pascal's triangle: C[n][r] = C[n−1][r−1] + C[n−1][r].

2.7 Euler's totient φ(n)

φ(n) = how many numbers in 1..n are coprime with n. φ(n) = n · Π (1 − 1/p) over primes p dividing n. Use: a^φ(m) ≡ 1 (mod m) when gcd(a, m) = 1, and counting fractions/pairs with gcd 1.

```
long long phi(long long n) {
    long long res = n;
    for (long long p = 2; p * p <= n; p++)
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;          // multiply by (1 - 1/p)
        }
    if (n > 1) res -= res / n;
    return res;
}
```

2.8 Inclusion–exclusion

|A ∪ B ∪ C| = |A| + |B| + |C| − |A∩B| − |A∩C| − |B∩C| + |A∩B∩C|. Loop over all subsets with a bitmask; odd-size subsets add, even-size subsets subtract.

Problem (CSES 2185 Prime Multiples): given n ≤ 10^18 and k ≤ 20 distinct primes, count numbers in 1..n divisible by at least one of the primes.

Approach: for each non-empty subset, product P of its primes; numbers divisible by all of them = n / P. Stop multiplying when P would exceed n (overflow guard).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ll n; int k; cin >> n >> k;
    vector<ll> p(k);
```

```
    for (auto& x : p) cin >> x;
    ll ans = 0;
    for (int mask = 1; mask < (1 << k); mask++) {
        ll prod = 1; bool over = false;
        for (int i = 0; i < k; i++) if (mask >> i & 1) {
            if (prod > n / p[i]) { over = true; break; }   // product > n ->
contributes 0
            prod *= p[i];
        }
        if (over) continue;
        if (__builtin_popcount(mask) % 2) ans += n / prod;  // odd size: add
        else ans -= n / prod;                               // even size:
subtract
    }
    cout << ans << "\n";
}
```

2.9 Matrix exponentiation

What it does: computes the n-th term of a linear recurrence in O(k^3 log n), where k is the matrix size. Works for n up to 10^18.

Problem (CSES 1722 Fibonacci Numbers): print F(n) mod 10^9+7 for n ≤ 10^18.

Approach: [F(n+1), F(n)] = M · [F(n), F(n−1)] with M = [[1,1],[1,0]]. So M^n gives F(n) in its top-right corner.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1e9 + 7;
using Mat = vector<vector<ll>>;

Mat mul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Mat C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++)
            for (int j = 0; j < m; j++)
                C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
    return C;
}
Mat matpow(Mat A, ll p) {
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;    // identity
    while (p) { if (p & 1) R = mul(R, A); A = mul(A, A); p >>= 1; }
    return R;
}

int main() {
    ll n; cin >> n;
    Mat M = {{1, 1}, {1, 0}};
```

```
    cout << matpow(M, n)[0][1] << "\n";
}
```

Other uses: count paths of length exactly k in a graph (adjacency matrix to the power k), dice combinations with huge n, any recurrence f(n) = c1·f(n−1) + … + ck·f(n−k).

Chapter 3. Prefix Sums, Binary Search, Two Pointers, Greedy

3.1 Prefix sums

What it does: after O(n) preprocessing, the sum of any range a[l..r] is answered in O(1).

pre[i] = a[0] + … + a[i−1], so sum(l..r) = pre[r+1] − pre[l].

Problem (CSES 1646 Static Range Sum Queries): array of n ≤ 2·10^5 numbers, q queries (a, b) 1-indexed; print the sum of positions a..b.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<ll> pre(n + 1, 0);
    for (int i = 1; i <= n; i++) { ll x; cin >> x; pre[i] = pre[i - 1] + x; }
    while (q--) {
        int a, b; cin >> a >> b;
        cout << pre[b] - pre[a - 1] << "\n";   // sum of a..b
    }
}
```

2D prefix sums. P[i][j] = sum of the rectangle (1,1)..(i,j).

- Build: P[i][j] = a[i][j] + P[i−1][j] + P[i][j−1] − P[i−1][j−1]
- Query (r1,c1)..(r2,c2) = P[r2][c2] − P[r1−1][c2] − P[r2][c1−1] + P[r1−1][c1−1]

(CSES 1652 Forest Queries: count trees in rectangles.)

Difference array (range update, point query). To add v to every element in [l, r]: d[l] += v, d[r+1] −= v. After all updates, the prefix sum of d gives the final array. O(1) per update.

Classic trick – subarrays with sum k: sum(l..r) = k means pre[r+1] − pre[l] = k. Walk left to right and count earlier prefixes equal to pre − k in a map.

```
// CSES 1661 Subarray Sums II: count subarrays with sum exactly x (values may be negative)
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; ll x; cin >> n >> x;
    map<ll, int> seen;
    seen[0] = 1;                    // empty prefix
    ll pre = 0, ans = 0;
    for (int i = 0; i < n; i++) {
```

```
        ll a; cin >> a;
        pre += a;
        ans += seen.count(pre - x) ? seen[pre - x] : 0;   // earlier prefix =
pre - x
        seen[pre]++;
    }
    cout << ans << "\n";
}
```

The same trick with `pre % n` (careful with negatives) solves "subarrays with sum divisible by n" (CSES 1662).

3.2 Binary search

What it does: finds a boundary in a MONOTONIC yes/no condition in O(log range) checks.

If check(x) is false, false, ..., false, true, true, ... then binary search finds the first true.

```
// first x in [lo, hi] with check(x) == true (assumes check(hi) is true)
long long lo = 0, hi = 2e18;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;   // avoids overflow
    if (check(mid)) hi = mid;             // mid works -> answer is mid or
smaller
    else lo = mid + 1;                    // mid fails -> answer is bigger
}
// lo is the answer
```

Binary search on the answer

Most binary search problems ask for "the minimum X such that something is possible". If X works, any larger X works too. Then binary search X, and write a greedy check(X).

Problem (CSES 1620 Factory Machines): n machines; machine i makes one product every k[i] seconds. What is the minimum time to make t products? (n ≤ 2·10^5, t, k ≤ 10^9.)

Approach: in time T machine i makes T / k[i] products. check(T) = sum of T / k[i] ≥ t. It is monotonic. Answer is at most min(k) · t ≤ 10^18. Stop summing once it reaches t to avoid overflow.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; ll t; cin >> n >> t;
    vector<ll> k(n);
    for (auto& x : k) cin >> x;
    auto check = [&](ll T) {
        ll made = 0;
        for (ll x : k) { made += T / x; if (made >= t) return true; }
        return false;
    };
```

```
    ll lo = 1, hi = *min_element(k.begin(), k.end()) * t;
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (check(mid)) hi = mid; else lo = mid + 1;
    }
    cout << lo << "\n";
}
```

Same pattern: "minimize the maximum" problems. Example: split an array into k parts minimizing the largest part sum (CSES 1085 Array Division) — check(S) = can we split into ≤ k parts each with sum ≤ S (greedy: start a new part when the next element does not fit).

Binary search on real numbers: loop 100 times instead of `while (lo < hi)`.

```
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (check(mid)) hi = mid; else lo = mid;
}
```

3.3 Two pointers and sliding window

What it does: moves a left pointer and a right pointer only forward, so the total work is O(n) instead of O(n^2).

Use it when: the array is sorted, or "the window [l, r] is valid" stays valid when it shrinks.

Problem (CSES 1141 Playlist): find the longest contiguous segment of songs with no repeated song.

Approach: extend r; while a[r] is already in the window, move l forward and remove a[l]. Track the best r − l + 1.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto& x : a) cin >> x;
    map<int, int> cnt;              // counts inside the window [l, r]
    int best = 0;
    for (int l = 0, r = 0; r < n; r++) {
        cnt[a[r]]++;
        while (cnt[a[r]] > 1) cnt[a[l++]]--;   // shrink until a[r] is unique
        best = max(best, r - l + 1);
    }
    cout << best << "\n";
}
```

Pair with sum x in a sorted array (CSES 1640 Sum of Two Values): l at the start, r at the end. If a[l] + a[r] < x move l right, if bigger move r left. Keep original indices when you sort.

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; long long x; cin >> n >> x;
    vector<pair<long long,int>> a(n);
    for (int i = 0; i < n; i++) { cin >> a[i].first; a[i].second = i + 1; }
    sort(a.begin(), a.end());
    int l = 0, r = n - 1;
    while (l < r) {
        long long s = a[l].first + a[r].first;
        if (s == x) { cout << min(a[l].second, a[r].second) << ' ' <<
max(a[l].second, a[r].second) << "\n"; return 0; }
        if (s < x) l++; else r--;
    }
    cout << "IMPOSSIBLE\n";
}
```

3.4 Greedy

Greedy = make the locally best choice, never undo it. It needs a reason (usually an exchange argument: "swapping to my choice never makes the answer worse"). Sorting is almost always the first step.

Problem type	Greedy rule
Max number of non-overlapping intervals (CSES 1629 Movie Festival)	Sort by END time, take each interval that starts after the last taken end
Min number of points to hit all intervals	Sort by end, put a point at the end of the first un-hit interval
Min platforms / rooms for intervals	Sort events; +1 at start, −1 at end; answer = max running count
Pair heavy with light (CSES 1090 Ferris Wheel)	Sort; two pointers: heaviest with lightest if it fits
Tasks with deadlines, maximize reward = deadline − finish time (CSES 1630)	Do shortest tasks first
Minimize sum of	a[i] − x
Coin change with "canonical" coins (1, 2, 5, 10…)	Take the largest coin first (NOT for arbitrary coins — use DP)
Smallest sum you cannot make from coins (CSES 2183 Missing Coin Sum)	Sort; keep reach = 1; if coin > reach stop, else reach += coin

Problem (CSES 1629 Movie Festival): n movies with start and end times; watch as many complete movies as possible (one at a time).

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
```

```
    vector<pair<int,int>> mv(n);              // stored as (end, start)
    for (auto& [e, s] : mv) cin >> s >> e;
    sort(mv.begin(), mv.end());               // earliest end first
    int cnt = 0, lastEnd = 0;
    for (auto [e, s] : mv)
        if (s >= lastEnd) { cnt++; lastEnd = e; }   // fits after the last
chosen movie
    cout << cnt << "\n";
}
```

Why end time? Picking the movie that ends first leaves the most time for the rest. Any optimal schedule can swap its first movie for ours without losing anything.

Problem (CSES 1091 Concert Tickets): n ticket prices, m customers in order with a maximum price each; each customer buys the most expensive ticket not above their maximum (or −1).

Approach: a multiset; for max price x, `upper_bound(x)` then step back one.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    multiset<int> t;
    for (int i = 0; i < n; i++) { int x; cin >> x; t.insert(x); }
    while (m--) {
        int x; cin >> x;
        auto it = t.upper_bound(x);           // first price > x
        if (it == t.begin()) { cout << -1 << "\n"; continue; }
        --it;                                 // largest price <= x
        cout << *it << "\n";
        t.erase(it);                          // erase THIS copy only
    }
}
```

3.5 Sorting tricks and sweep line

- **Coordinate compression:** values up to 10^9 but only n ≤ 2·10^5 of them → sort the unique values and replace each value by its index (`lower_bound`). Then you can use arrays / Fenwick trees over indices.
- **Sweep line:** turn intervals into events (time, +1) and (time, −1), sort, and scan. Max overlap = max prefix sum (CSES 1619 Restaurant Customers).
- **Custom sort:** `sort(all(v), [](auto& a, auto& b){ return a.second < b.second; });` The comparator must be strict (use `<`, never `<=`).

```
// coordinate compression
vector<int> vals = a;
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
for (int& x : a) x = lower_bound(vals.begin(), vals.end(), x) - vals.begin(); //
0..k-1
```

```cpp
// CSES 1619 Restaurant Customers: max customers present at the same time
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<pair<int,int>> ev;
    for (int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        ev.push_back({a, +1});   // arrival
        ev.push_back({b, -1});   // leaving
    }
    sort(ev.begin(), ev.end());  // at equal time, -1 comes before +1
    int cur = 0, best = 0;
    for (auto [t, d] : ev) { cur += d; best = max(best, cur); }
    cout << best << "\n";
}
```

3.6 Meet in the middle

When n ≤ 40, 2^40 is too many subsets, but 2^20 is fine. Split items into two halves, list all subset sums of each half, sort one list and binary search (or two pointers) for matches.

Problem (CSES 1628 Meet in the Middle): count subsets of n ≤ 40 numbers with sum exactly x.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
vector<ll> sums(const vector<ll>& v) {
    int k = v.size();
    vector<ll> s(1 << k, 0);
    for (int mask = 1; mask < (1 << k); mask++) {
        int low = __builtin_ctz(mask);              // lowest bit
        s[mask] = s[mask ^ (1 << low)] + v[low];   // build from a smaller
subset
    }
    return s;
}
int main() {
    int n; ll x; cin >> n >> x;
    vector<ll> a(n);
    for (auto& v : a) cin >> v;
    vector<ll> L(a.begin(), a.begin() + n / 2), R(a.begin() + n / 2, a.end());
    vector<ll> sl = sums(L), sr = sums(R);
    sort(sr.begin(), sr.end());
    ll ans = 0;
    for (ll s : sl)                                 // need s + t == x
        ans += upper_bound(sr.begin(), sr.end(), x - s) -
lower_bound(sr.begin(), sr.end(), x - s);
    cout << ans << "\n";
}
```

Chapter 4. Dynamic Programming

4.1 How to think about DP

DP = recursion + remembering answers. Use it when the problem has **overlapping subproblems** (the same smaller question is asked many times) and **optimal substructure** (the best answer is built from best answers of smaller parts).

Four questions to answer before coding:

1. **State:** what does dp[...] mean, in one sentence? ("dp[i] = number of ways to make sum i")
2. **Transition:** how is dp[state] computed from smaller states?
3. **Base case:** the smallest states (dp[0] = 1 ...).
4. **Order / answer:** in which order to fill the table, and which entry is the answer?

Complexity = (number of states) × (work per transition).

Two styles:

- **Top-down (memoization):** write the recursion, cache results. Easy to write; recursion can be deep.
- **Bottom-up (tabulation):** loops over states in the right order. Faster, no stack issue. Preferred in contests.

4.2 Counting ways: Dice Combinations

Problem (CSES 1633): count the ways to get sum n (n ≤ 10^6) by throwing a die one or more times (order matters), mod 10^9+7.

State: dp[s] = number of ordered ways to make sum s. **Transition:** the last throw is 1..6, so dp[s] = dp[s−1] + ... + dp[s−6]. **Base:** dp[0] = 1.

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
int main() {
    int n; cin >> n;
    vector<long long> dp(n + 1, 0);
    dp[0] = 1;
    for (int s = 1; s <= n; s++)
        for (int d = 1; d <= 6 && d <= s; d++)
            dp[s] = (dp[s] + dp[s - d]) % MOD;
    cout << dp[n] << "\n";
}
```

4.3 Coin problems (the three classic versions)

Coins c1..cn, target x.

Question	State and transition	Loop order
Minimum number of coins (CSES 1634)	dp[s] = min over c of dp[s−c] + 1	s outer
Number of ORDERED ways (CSES 1635)	dp[s] = Σ dp[s−c]	s outer, coin inner
Number of UNORDERED ways (CSES 1636)	dp[s] += dp[s−c]	coin OUTER, s inner

The loop order is the difference between counting {1,2} and {2,1} once or twice. With the coin loop outside, each combination is built in one fixed coin order, so it is counted once.

```cpp
// Minimizing Coins (CSES 1634)
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x; cin >> n >> x;
    vector<int> c(n);
    for (auto& v : c) cin >> v;
    const int INF = 1e9;
    vector<int> dp(x + 1, INF);
    dp[0] = 0;
    for (int s = 1; s <= x; s++)
        for (int coin : c)
            if (coin <= s && dp[s - coin] != INF)
                dp[s] = min(dp[s], dp[s - coin] + 1);
    cout << (dp[x] == INF ? -1 : dp[x]) << "\n";
}
```

```cpp
// Coin Combinations II (CSES 1636): unordered ways
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
int main() {
    int n, x; cin >> n >> x;
    vector<int> c(n);
    for (auto& v : c) cin >> v;
    vector<int> dp(x + 1, 0);
    dp[0] = 1;
    for (int coin : c)                      // coin loop OUTSIDE
        for (int s = coin; s <= x; s++)
            dp[s] = (dp[s] + dp[s - coin]) % MOD;
    cout << dp[x] << "\n";
}
```

4.4 Grid DP

Problem (CSES 1638 Grid Paths): n×n grid with traps '*'; count paths from top-left to bottom-right moving only right or down, mod 10^9+7.

State: dp[i][j] = paths reaching (i, j). **Transition:** dp[i][j] = dp[i−1][j] + dp[i][j−1], or 0 on a trap.

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
int main() {
    int n; cin >> n;
    vector<string> g(n);
    for (auto& s : g) cin >> s;
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            if (g[i][j] == '*') { dp[i][j] = 0; continue; }
            if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
            long long v = 0;
            if (i > 0) v += dp[i - 1][j];
            if (j > 0) v += dp[i][j - 1];
            dp[i][j] = v % MOD;
        }
    cout << dp[n - 1][n - 1] << "\n";
}
```

Variants: maximum-sum path (use max instead of +), minimum cost path, paths with obstacles given as a list (sort them and use inclusion–exclusion with nCr when the grid is huge).

4.5 0/1 Knapsack

Problem (CSES 1158 Book Shop): n books with price h[i] and pages s[i]; budget x. Maximize total pages, each book at most once. (n ≤ 1000, x ≤ 10^5.)

State: dp[c] = max pages with total price at most c using the books seen so far. **Transition:** for each book, dp[c] = max(dp[c], dp[c − h] + s). **Key detail:** loop c DOWNWARD so each book is used at most once (going upward would reuse the same book — that is the unbounded knapsack).

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, x; cin >> n >> x;
    vector<int> h(n), s(n);
    for (auto& v : h) cin >> v;
    for (auto& v : s) cin >> v;
    vector<int> dp(x + 1, 0);
    for (int i = 0; i < n; i++)
        for (int c = x; c >= h[i]; c--)              // DOWNWARD: each item
once
            dp[c] = max(dp[c], dp[c - h[i]] + s[i]);
    cout << dp[x] << "\n";
}
```

Knapsack variant	Change
Unbounded (items reusable)	Loop c UPWARD
Values small, weights huge	dp[value] = min weight to get that value
Subset sum yes/no	`bitset<MAXS> b; b[0]=1; for v: b |= b << v;` (64× faster)
Count subsets with sum S	dp[c] += dp[c − v], downward
Split into two parts with equal sum (CSES 1093)	Count subsets with sum total/2, divide by 2

4.6 Longest Increasing Subsequence (LIS) in O(n log n)

Problem (CSES 1145): length of the longest strictly increasing subsequence, n ≤ 2·10^5.

Approach: keep `tail[k]` = the smallest possible last value of an increasing subsequence of length k+1. For each x, find the first tail ≥ x (lower_bound) and replace it; if none, append. The size of `tail` is the answer. (For non-decreasing, use upper_bound.)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> tail;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        auto it = lower_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);   // extends the longest one
        else *it = x;                              // better (smaller) tail for
that length
    }
    cout << tail.size() << "\n";
}
```

The O(n^2) version (dp[i] = 1 + max dp[j] for j < i with a[j] < a[i]) is easier to adapt to weighted variants.

4.7 LCS and Edit Distance (two strings)

LCS: dp[i][j] = longest common subsequence of a[0..i) and b[0..j). - If a[i−1] == b[j−1]: dp[i][j] = dp[i−1][j−1] + 1 - else dp[i][j] = max(dp[i−1][j], dp[i][j−1])

Edit distance (CSES 1639): minimum insert / delete / replace operations to turn a into b. - dp[i][0] = i, dp[0][j] = j - dp[i][j] = min(dp[i−1][j] + 1 (delete), dp[i][j−1] + 1 (insert), dp[i−1][j−1] + (a[i−1] != b[j−1]) (replace or match))

```
#include <bits/stdc++.h>
using namespace std;
int main() {
```

```
    string a, b; cin >> a >> b;
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = min({dp[i - 1][j] + 1,                          // delete
a[i-1]
                            dp[i][j - 1] + 1,                          // insert
b[j-1]
                            dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])}); //
replace / match
    cout << dp[n][m] << "\n";
}
```

4.8 Interval DP

State: dp[l][r] = best answer for the segment l..r. Fill by increasing length. Typical: O(n^2) states and O(1) or O(n) transitions.

Problem (CSES 1097 Removal Game): n numbers in a row; two players alternately take the first or last number; both maximize their own sum. Print the first player's maximum score.

Approach: dp[l][r] = best (my score − opponent score) on a[l..r]. dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]). First player's score = (total + dp[0][n−1]) / 2.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; cin >> n;
    vector<ll> a(n);
    ll total = 0;
    for (auto& x : a) { cin >> x; total += x; }
    vector<vector<ll>> dp(n, vector<ll>(n, 0));
    for (int l = n - 1; l >= 0; l--) {
        dp[l][l] = a[l];
        for (int r = l + 1; r < n; r++)
            dp[l][r] = max(a[l] - dp[l + 1][r], a[r] - dp[l][r - 1]);
    }
    cout << (total + dp[0][n - 1]) / 2 << "\n";
}
```

Other interval DP problems: matrix chain multiplication, merging stones / slimes (dp[l][r] = min over k of dp[l][k] + dp[k+1][r] + sum(l..r)), palindrome partitioning, burst balloons.

4.9 Bitmask DP

Use when n ≤ 20 and the state is "which items are already used". See the separate template file; the core shape:

```
// dp[mask] = best value when exactly the items in mask are used
for (int mask = 0; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++)
        if (!(mask >> i & 1))                                  // item i not used
yet
            dp[mask | (1 << i)] = better(dp[mask | (1 << i)], extend(dp[mask],
i));
```

Problem (CSES 1690 Hamiltonian Flights): count routes from city 1 to city n visiting every city exactly once (n ≤ 20, directed flights).

State: dp[mask][v] = routes that visited exactly mask and stand at v.

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
int dp[1 << 20][20];
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> in(n);                    // in[v] = cities with a
flight to v
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; in[b - 1].push_back(a
- 1); }
    dp[1][0] = 1;
    for (int mask = 1; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;                        // must contain the
start
        if ((mask >> (n - 1) & 1) && mask != (1 << n) - 1) continue; // end city
only at the very end
        for (int v = 0; v < n; v++) {
            if (!(mask >> v & 1)) continue;
            int prev = mask ^ (1 << v);
            if (prev == 0) continue;
            for (int u : in[v])
                if (prev >> u & 1) dp[mask][v] = (dp[mask][v] + dp[prev][u]) %
MOD;
        }
    }
    cout << dp[(1 << n) - 1][n - 1] << "\n";
}
```

4.10 Digit DP

When: "count numbers in [L, R] whose digits satisfy something", with R up to 10^18. Answer = f(R) − f(L − 1), where f(X) counts in [0, X].

State: (position, tight, extra info). *tight* = are we still equal to X's prefix (then the next digit is limited by X's digit)? Add a *started* flag when leading zeros matter.

Problem (CSES 2220 Counting Numbers): count numbers in [a, b] with no two adjacent equal digits.

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
using ll = long long;
string s;
ll memo[20][11][2][2];
bool vis[20][11][2][2];

// pos: current digit index, prev: previous digit (10 = none), tight, started
ll go(int pos, int prev, bool tight, bool started) {
    if (pos == (int)s.size()) return 1;                 // built a valid number
    if (vis[pos][prev][tight][started]) return memo[pos][prev][tight][started];
    int limit = tight ? s[pos] - '0' : 9;
    ll res = 0;
    for (int d = 0; d <= limit; d++) {
        if (started && d == prev) continue;             // adjacent equal digits
not allowed
        bool ns = started || d != 0;                    // leading zeros do not
count as digits
        res += go(pos + 1, ns ? d : 10, tight && d == limit, ns);
    }
    vis[pos][prev][tight][started] = true;
    return memo[pos][prev][tight][started] = res;
}
ll f(ll x) {                                            // count in [0, x]
    if (x < 0) return 0;
    s = to_string(x);
    memset(vis, 0, sizeof vis);
    return go(0, 10, true, false);
}
int main() {
    ll a, b; cin >> a >> b;
    cout << f(b) - f(a - 1) << "\n";
}
```

Other digit DP questions: sum of digits in a range, numbers divisible by k whose digit sum is also divisible by k (state adds remainder), count of a digit d in all numbers up to X.

4.11 DP on trees

Root the tree. dp[v] is computed from dp of v's children (post-order DFS).

Problem (CSES 1130 Tree Matching): maximum number of edges with no shared vertex in a tree.

State: dp[v][0] = best in v's subtree when v is NOT matched; dp[v][1] = best when v IS matched with one child. - dp[v][0] = Σ max(dp[c][0], dp[c][1]) - dp[v][1] = max over child c of dp[v][0] − max(dp[c][0], dp[c][1]) + dp[c][0] + 1

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b;
adj[a].push_back(b); adj[b].push_back(a); }
```

```
    // iterative DFS order so deep trees do not overflow the stack
    vector<int> order, par(n + 1, 0), st = {1};
    par[1] = -1;
    while (!st.empty()) {
        int v = st.back(); st.pop_back();
        order.push_back(v);
        for (int u : adj[v]) if (u != par[v]) { par[u] = v; st.push_back(u); }
    }
    vector<array<int,2>> dp(n + 1, {0, 0});
    for (int i = n - 1; i >= 0; i--) {                // children before
parents
        int v = order[i];
        for (int c : adj[v]) if (c != par[v]) dp[v][0] += max(dp[c][0], dp[c]
[1]);
        for (int c : adj[v]) if (c != par[v])
            dp[v][1] = max(dp[v][1], dp[v][0] - max(dp[c][0], dp[c][1]) + dp[c]
[0] + 1);
    }
    cout << max(dp[1][0], dp[1][1]) << "\n";
}
```

Rerooting (all roots in O(n)): compute dp "down" with one DFS, then pass the parent's contribution "up" with a second DFS. Example: sum of distances from every node to all others (CSES 1133 Tree Distances II): ans[child] = ans[v] − size[child] + (n − size[child]).

4.12 DP on a DAG

If the graph has no cycles, process nodes in topological order: dp[v] = best over edges (u → v) of dp[u] + w.

- Longest path from 1 to n (CSES 1680 Longest Flight Route): dp = max, keep parent for the path.
- Number of paths from 1 to n (CSES 1681 Game Routes): dp[v] = Σ dp[u].
- Shortest path counting with Dijkstra (CSES 1202): count, min and max number of edges along shortest routes.

```
// CSES 1681 Game Routes: number of paths 1 -> n in a DAG
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1e9 + 7;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
indeg[b]++; }
    queue<int> q;
    for (int v = 1; v <= n; v++) if (indeg[v] == 0) q.push(v);
    vector<long long> ways(n + 1, 0);
    ways[1] = 1;
    while (!q.empty()) {                    // Kahn's topological order
```

```
        int v = q.front(); q.pop();
        for (int u : adj[v]) {
            ways[u] = (ways[u] + ways[v]) % MOD;
            if (--indeg[u] == 0) q.push(u);
        }
    }
    cout << ways[n] << "\n";
}
```

4.13 DP optimization notes

- **Memory:** if dp[i] only uses dp[i−1], keep two rows (or one row with the right loop direction).
- **Prefix sums inside DP:** if dp[i] = Σ dp[j] for j in a range, maintain a prefix sum array → O(1) per state.
- **Monotonic deque:** dp[i] = min(dp[j]) for j in a sliding window [i−k, i−1] → O(n) total.
- **Binary search in DP:** sort intervals by end; dp[i] = max(dp[i−1], value[i] + dp[last interval ending before start i]) (CSES 1140 Projects, weighted interval scheduling).

```
// CSES 1140 Projects: weighted interval scheduling
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; cin >> n;
    vector<array<ll,3>> p(n);                        // {end, start, reward}
    for (auto& [e, s, r] : p) cin >> s >> e >> r;
    sort(p.begin(), p.end());
    vector<ll> ends(n), dp(n + 1, 0);                // dp[i] = best using the
first i projects
    for (int i = 0; i < n; i++) ends[i] = p[i][0];
    for (int i = 0; i < n; i++) {
        // number of projects that end strictly before this one starts
        int j = lower_bound(ends.begin(), ends.end(), p[i][1]) - ends.begin();
        dp[i + 1] = max(dp[i], dp[j] + p[i][2]);    // skip it, or take it
    }
    cout << dp[n] << "\n";
}
```

Chapter 5. Data Structures

5.1 Monotonic stack

What it does: for every element, finds the nearest element to the left (or right) that is smaller (or larger), in O(n) total.

Idea: keep a stack of indices whose values are increasing. Before pushing i, pop everything ≥ a[i] — those can never be the "nearest smaller" for anyone to the right of i.

Problem (CSES 1645 Nearest Smaller Values): for each position, print the position of the nearest smaller value to its left (0 if none).

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    stack<int> st;                          // indices, values increasing from
bottom to top
    for (int i = 1; i <= n; i++) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        cout << (st.empty() ? 0 : st.top()) << (i == n ? '\n' : ' ');
        st.push(i);
    }
}
```

Uses: largest rectangle in a histogram (for each bar, nearest smaller on both sides gives its width), stock span, "next greater element", counting subarrays where a[i] is the minimum.

5.2 Sliding window maximum (monotonic deque)

Keep indices in a deque with values decreasing. The front is the maximum of the current window. Pop the front when it leaves the window; pop the back while it is smaller than the new element. O(n) total.

```
vector<int> slidingMax(const vector<int>& a, int k) {
    deque<int> dq;                                  // indices, a[] decreasing
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();    // left the window
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

5.3 Fenwick tree (Binary Indexed Tree)

What it does: point update and prefix sum query, both O(log n). Short to code, very fast.

How: index i stores the sum of a block of length (i & −i) ending at i.

Problem (CSES 1648 Dynamic Range Sum Queries): array of n numbers; queries "1 k u" set a[k] = u, "2 a b" print sum of a..b.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Fenwick {
    int n; vector<ll> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, ll v) { for (; i <= n; i += i & -i) t[i] += v; }     // a[i] += v (1-indexed)
    ll sum(int i) { ll s = 0; for (; i > 0; i -= i & -i) s += t[i]; return s; } // a[1..i]
    ll sum(int l, int r) { return sum(r) - sum(l - 1); }
};
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<ll> a(n + 1);
    Fenwick fw(n);
    for (int i = 1; i <= n; i++) { cin >> a[i]; fw.add(i, a[i]); }
    while (q--) {
        int type; cin >> type;
        if (type == 1) { int k; ll u; cin >> k >> u; fw.add(k, u - a[k]); a[k] = u; }  // set = add the difference
        else { int l, r; cin >> l >> r; cout << fw.sum(l, r) << "\n"; }
    }
}
```

Tricks with Fenwick:

- **Range update, point query:** store the difference array: add(l, v), add(r+1, −v); a[i] = sum(i).
- **Count inversions:** go left to right, for each x add (number of earlier values > x) = i − sum(x); then add(x, 1). Compress values first.
- **k-th smallest / order statistics:** binary lifting on the Fenwick tree, or use the ordered set below.

5.4 Segment tree

What it does: any associative operation (sum, min, max, gcd, xor, ...) on a range, with point updates. O(log n) each.

Problem (CSES 1649 Dynamic Range Minimum Queries): queries "1 k u" set a[k] = u, "2 a b" print the minimum of a..b.

```cpp
#include <bits/stdc++.h>
using namespace std;
// Iterative (bottom-up) segment tree: short and fast.  0-indexed, query on [l, r).
struct SegTree {
    int n; vector<int> t;
    const int NEUTRAL = INT_MAX;                    // identity of min
    int op(int a, int b) { return min(a, b); }      // change for sum / max / gcd ...
    SegTree(int n) : n(n), t(2 * n, INT_MAX) {}
    void set(int p, int v) {
        for (t[p += n] = v; p > 1; p >>= 1) t[p >> 1] = op(t[p], t[p ^ 1]);
    }
    int query(int l, int r) {                       // op over [l, r)
        int resL = NEUTRAL, resR = NEUTRAL;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) resL = op(resL, t[l++]);
            if (r & 1) resR = op(t[--r], resR);
        }
        return op(resL, resR);
    }
};
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    SegTree st(n);
    for (int i = 0; i < n; i++) { int x; cin >> x; st.set(i, x); }
    while (q--) {
        int type, a, b; cin >> type >> a >> b;
        if (type == 1) st.set(a - 1, b);
        else cout << st.query(a - 1, b) << "\n";    // 1-indexed [a, b] -> [a-1, b)
    }
}
```

What to store in a node (common problems):

Query	Node stores	Merge
Max subarray sum in a range (CSES 1190)	sum, best prefix, best suffix, best	best = max(L.best, R.best, L.suf + R.pre)
Count of minimum	(min, count)	equal mins add counts
First index with value ≥ x (CSES 1143 Hotel Queries)	max	walk down: go left if left max ≥ x
Number of distinct values in ranges (offline)	sort queries by r, keep last occurrence	Fenwick is enough

5.5 Lazy segment tree (range update + range query)

When updates affect a whole range ("add v to a[l..r]"), store a pending update (**lazy tag**) in the node and push it to the children only when needed.

Problem (range add, range sum): queries "1 l r v" add v to a[l..r], "2 l r" print sum of a[l..r].

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct LazySeg {
    int n; vector<ll> sum, lz;
    LazySeg(int n) : n(n), sum(4 * n, 0), lz(4 * n, 0) {}
    void apply(int node, int len, ll v) { sum[node] += v * len; lz[node] += v; }
    void push(int node, int l, int r) {                    // give the pending
add to children
        if (lz[node] == 0) return;
        int m = (l + r) / 2;
        apply(2 * node, m - l + 1, lz[node]);
        apply(2 * node + 1, r - m, lz[node]);
        lz[node] = 0;
    }
    void update(int node, int l, int r, int ql, int qr, ll v) {
        if (qr < l || r < ql) return;                       // no overlap
        if (ql <= l && r <= qr) { apply(node, r - l + 1, v); return; } // fully
inside
        push(node, l, r);
        int m = (l + r) / 2;
        update(2 * node, l, m, ql, qr, v);
        update(2 * node + 1, m + 1, r, ql, qr, v);
        sum[node] = sum[2 * node] + sum[2 * node + 1];
    }
    ll query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return sum[node];
        push(node, l, r);
        int m = (l + r) / 2;
        return query(2 * node, l, m, ql, qr) + query(2 * node + 1, m + 1, r, ql,
qr);
    }
    void update(int l, int r, ll v) { update(1, 0, n - 1, l, r, v); }
    ll query(int l, int r) { return query(1, 0, n - 1, l, r); }
};
int main() {
    int n, q; cin >> n >> q;
    LazySeg st(n);
    for (int i = 0; i < n; i++) { ll x; cin >> x; st.update(i, i, x); }
    while (q--) {
        int type; cin >> type;
        if (type == 1) { int l, r; ll v; cin >> l >> r >> v; st.update(l - 1, r
- 1, v); }
        else { int l, r; cin >> l >> r; cout << st.query(l - 1, r - 1) << "\n";
}
    }
}
```

For "assign v to a range" use a tag with a flag (has assignment) instead of an add. For both assign and add (CSES 1735 Range Updates and Sums), assignment must clear the pending add.

5.6 Sparse table (static range min/max in O(1))

If the array never changes, precompute mn[j][i] = min of a[i .. i + 2^j − 1] in O(n log n). A query [l, r] is covered by two overlapping blocks of length 2^k where k = log2(r − l + 1). Works for min, max, gcd (idempotent operations), NOT for sum.

```
struct SparseTable {
    vector<vector<int>> mn; vector<int> lg;
    SparseTable(const vector<int>& a) {
        int n = a.size(); lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        mn.assign(lg[n] + 1, a);
        for (int j = 1; j <= lg[n]; j++)
            for (int i = 0; i + (1 << j) <= n; i++)
                mn[j][i] = min(mn[j - 1][i], mn[j - 1][i + (1 << (j - 1))]);
    }
    int query(int l, int r) {                  // min of a[l..r], inclusive
        int k = lg[r - l + 1];
        return min(mn[k][l], mn[k][r - (1 << k) + 1]);
    }
};
```

5.7 Ordered set (policy-based data structure)

GCC has a balanced tree that also answers "k-th smallest" and "how many are smaller than x" in O(log n).

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

// ordered_set<int> s;  s.insert(5);
// *s.find_by_order(k)   -> k-th smallest (0-indexed)
// s.order_of_key(x)     -> number of elements < x
// For duplicates, store pairs (value, unique id).
```

Problem (CSES 2162 Josephus Problem II): children 1..n in a circle; skip k children, remove the next one; print the removal order. With an ordered set: pos = (pos + k) % size, print find_by_order(pos), erase it. O(n log n).

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> ordered_set;
int main() {
    int n, k; cin >> n >> k;
    ordered_set s;
```

```
    for (int i = 1; i <= n; i++) s.insert(i);
    int pos = 0;
    for (int i = 0; i < n; i++) {
        pos = (pos + k) % s.size();
        auto it = s.find_by_order(pos);
        cout << *it << (i == n - 1 ? '\n' : ' ');
        s.erase(it);                    // next child takes this position
    }
}
```

5.8 DSU (Union-Find) in practice

See the graph chapter for the code. Typical problems:

- **Road Construction (CSES 1676):** after each new road, print the number of components and the size of the largest. Each successful union reduces components by one.
- **Offline deletions:** DSU cannot delete edges. Process deletions in reverse as additions.
- **DSU with extra data:** store sum / min / parity in the root (bipartiteness with a parity bit).

5.9 Square root decomposition and Mo's algorithm

- **Sqrt decomposition:** split the array into blocks of size about √n; keep a summary per block. Query = whole blocks + partial ends → O(√n).
- **Mo's algorithm (offline range queries):** sort queries by (l / B, r), move two pointers and add/remove elements one at a time. Total O((n + q)√n). Use it for "number of distinct values in [l, r]" type queries when there are no updates.

```
// Mo's skeleton
int B = max(1, (int)sqrt(n));
sort(qs.begin(), qs.end(), [&](const Query& a, const Query& b) {
    if (a.l / B != b.l / B) return a.l / B < b.l / B;
    return (a.l / B & 1) ? a.r > b.r : a.r < b.r;   // zig-zag: faster
});
int L = 0, R = -1;                      // current window [L, R]
for (auto& q : qs) {
    while (R < q.r) add(++R);
    while (L > q.l) add(--L);
    while (R > q.r) remove(R--);
    while (L < q.l) remove(L++);
    ans[q.id] = currentAnswer;
}
```

Chapter 6. Strings

6.1 String basics in C++

```
string s; cin >> s;                 // one word
getline(cin, s);                    // whole line (after cin >> x, call
cin.ignore() first)
s.substr(pos, len);                 // O(len) copy — avoid inside big loops
s.find("abc");                      // O(n*m) worst case; returns string::npos
if missing
reverse(s.begin(), s.end());
int c = s[i] - 'a';                 // letter -> 0..25
char ch = 'a' + c;
to_string(123); stoll("123");
```

Counting characters: `int cnt[26] = {}; for (char c : s) cnt[c - 'a']++;` — anagram checks, "can you rearrange into a palindrome" (at most one odd count).

6.2 Prefix function and KMP

Prefix function: pi[i] = length of the longest proper prefix of s[0..i] that is also a suffix of s[0..i]. Computed in O(n).

Pattern matching (KMP): compute pi on `pattern + '#' + text`. Every position where pi equals |pattern| is an occurrence. O(n + m).

Problem (CSES 1753 String Matching): count occurrences of pattern p in text s.

```
#include <bits/stdc++.h>
using namespace std;
vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];                      // try to extend the previous
border
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];  // fall back to a shorter
border
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
int main() {
    string s, p; cin >> s >> p;
    string t = p + "#" + s;                     // '#' never appears in the
strings
    vector<int> pi = prefixFunction(t);
    int cnt = 0;
    for (int i = p.size() + 1; i < (int)t.size(); i++)
        if (pi[i] == (int)p.size()) cnt++;
```

```
    cout << cnt << "\n";
}
```

More uses of pi:

- Shortest period of s: k = n − pi[n−1]. If n % k == 0, s is (s[0..k)) repeated n/k times.
- All borders (prefixes that are also suffixes, CSES 1732 Finding Borders): pi[n−1], pi[pi[n−1]−1], ...

6.3 Z-function

z[i] = length of the longest common prefix of s and s[i..]. O(n). Same power as the prefix function; some people find it easier.

```
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);          // reuse the known box
[l, r)
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
// occurrences of p in s: z on p + "#" + s, positions with z[i] == |p|
```

6.4 String hashing

Idea: turn a string into a number h = s[0]·B^(n−1) + ... + s[n−1] (mod M). Equal strings have equal hashes; different strings collide with tiny probability. With prefix hashes, any substring's hash is O(1).

Compare substrings in O(1), find the longest repeated substring with binary search, count distinct substrings of a fixed length, check palindromes (hash of s vs reversed s).

```
// Double hashing (two moduli) to make collisions practically impossible
struct Hash {
    static const long long M1 = 1e9 + 7, M2 = 1e9 + 9, B = 131;
    vector<long long> h1, h2, p1, p2;
    Hash(const string& s) {
        int n = s.size();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0); p1.assign(n + 1, 1);
p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * B + s[i]) % M1;
            h2[i + 1] = (h2[i] * B + s[i]) % M2;
            p1[i + 1] = p1[i] * B % M1;
            p2[i + 1] = p2[i] * B % M2;
        }
    }
```

```
    // hash of s[l..r), as a pair
    pair<long long,long long> get(int l, int r) {
        long long a = (h1[r] - h1[l] * p1[r - l]) % M1; if (a < 0) a += M1;
        long long b = (h2[r] - h2[l] * p2[r - l]) % M2; if (b < 0) b += M2;
        return {a, b};
    }
};
```

Problem: longest substring that appears at least twice. **Approach:** if length L works, L−1 works too, so binary search L; check(L) = put all hashes of length L in a set and look for a duplicate. O(n log n).

```
#include <bits/stdc++.h>
using namespace std;
struct Hash {
    static const long long M1 = 1e9 + 7, M2 = 1e9 + 9, B = 131;
    vector<long long> h1, h2, p1, p2;
    Hash(const string& s) {
        int n = s.size();
        h1.assign(n + 1, 0); h2.assign(n + 1, 0); p1.assign(n + 1, 1);
p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * B + s[i]) % M1; h2[i + 1] = (h2[i] * B + s[i])
% M2;
            p1[i + 1] = p1[i] * B % M1;          p2[i + 1] = p2[i] * B % M2;
        }
    }
    long long get(int l, int r) {                 // both hashes packed in one
64-bit number
        long long a = ((h1[r] - h1[l] * p1[r - l]) % M1 + M1) % M1;
        long long b = ((h2[r] - h2[l] * p2[r - l]) % M2 + M2) % M2;
        return a * M2 + b;
    }
};
int main() {
    string s; cin >> s;
    int n = s.size();
    Hash h(s);
    auto check = [&](int L) {
        unordered_set<long long> seen;
        for (int i = 0; i + L <= n; i++)
            if (!seen.insert(h.get(i, i + L)).second) return true;   // already
seen
        return false;
    };
    int lo = 0, hi = n - 1;                       // find the largest L with
check(L)
    while (lo < hi) {
        int mid = (lo + hi + 1) / 2;
        if (check(mid)) lo = mid; else hi = mid - 1;
    }
    cout << lo << "\n";
}
```

6.5 Trie

What it does: a tree of characters storing many strings. Insert and search in O(length). Shares common prefixes.

Uses: prefix counting ("how many words start with abc"), word break DP (CSES 1731 Word Combinations), and the **binary trie** for maximum XOR pairs.

Problem (maximum XOR of two numbers / CSES 1655 Maximum Xor Subarray): max XOR of a subarray = max over i < j of pre[i] XOR pre[j]. Insert prefix XORs into a binary trie; for each new prefix, walk greedily toward the opposite bit.

```
#include <bits/stdc++.h>
using namespace std;
struct BinaryTrie {
    vector<array<int,2>> nxt;
    BinaryTrie() { nxt.push_back({0, 0}); }       // node 0 = root (0 also means
"no child")
    void insert(int x) {
        int cur = 0;
        for (int b = 30; b >= 0; b--) {
            int bit = x >> b & 1;
            if (!nxt[cur][bit]) { nxt[cur][bit] = nxt.size(); nxt.push_back({0,
0}); }
            cur = nxt[cur][bit];
        }
    }
    int maxXor(int x) {                           // max of x ^ y over inserted
y
        int cur = 0, res = 0;
        for (int b = 30; b >= 0; b--) {
            int want = (x >> b & 1) ^ 1;          // opposite bit gives a 1 in
the XOR
            if (nxt[cur][want]) { res |= 1 << b; cur = nxt[cur][want]; }
            else cur = nxt[cur][want ^ 1];
        }
        return res;
    }
};
int main() {
    int n; cin >> n;
    BinaryTrie t;
    t.insert(0);                                  // empty prefix
    int pre = 0, best = 0;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        pre ^= x;
        best = max(best, t.maxXor(pre));
        t.insert(pre);
    }
    cout << best << "\n";
}
```

Character trie for words:

```
struct Trie {
    vector<array<int,26>> nxt; vector<int> endCnt, passCnt;
    Trie() { newNode(); }
    int newNode() { nxt.push_back({}); nxt.back().fill(0); endCnt.push_back(0);
passCnt.push_back(0); return nxt.size() - 1; }
    void insert(const string& w) {
        int cur = 0;
        for (char ch : w) {
            int c = ch - 'a';
            if (!nxt[cur][c]) { int id = newNode(); nxt[cur][c] = id; }
            cur = nxt[cur][c];
            passCnt[cur]++;                        // words passing through this
prefix
        }
        endCnt[cur]++;                             // words ending here
    }
    int countPrefix(const string& p) {
        int cur = 0;
        for (char ch : p) { int c = ch - 'a'; if (!nxt[cur][c]) return 0; cur =
nxt[cur][c]; }
        return passCnt[cur];
    }
};
```

6.6 Palindromes and Manacher

- Check palindrome: compare s with its reverse, or two pointers.
- **Longest palindromic substring in O(n^2):** expand around each center (2n−1 centers). Fine for n ≤ 5000.
- **Manacher, O(n):** d1[i] = number of odd palindromes centered at i (radius), d2[i] for even ones. Used in CSES 1111 Longest Palindrome.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s; cin >> s;
    // Manacher on t = #s[0]#s[1]#...# so odd and even palindromes are handled
together
    string t = "#";
    for (char c : s) { t += c; t += '#'; }
    int n = t.size();
    vector<int> d(n, 0);                           // d[i] = radius of the
palindrome centered at i in t
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 1 : min(d[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && t[i - k] == t[i + k]) k++;
        d[i] = k--;
        if (i + k > r) { l = i - k; r = i + k; }
    }
    int best = 0, center = 0;
    for (int i = 0; i < n; i++) if (d[i] - 1 > best) { best = d[i] - 1; center =
i; }
    // palindrome length in s = d[i] - 1; start index in s:
```

```
    int start = (center - best) / 2;
    cout << s.substr(start, best) << "\n";
}
```

6.7 Quick reference: which string tool?

Task	Tool	Time
Find all occurrences of a pattern	KMP / Z-function	O(n + m)
Compare any two substrings many times	Hashing	O(1) per compare
Longest repeated / common substring	Hashing + binary search (or suffix array)	O(n log n)
Many patterns / prefix queries	Trie (Aho-Corasick for many patterns in a text)	O(total length)
Max XOR pair	Binary trie	O(n · 30)
Longest palindrome	Manacher	O(n)
Period / borders	Prefix function	O(n)
Smallest rotation	Booth's algorithm or s+s with hashing	O(n) / O(n log n)

Chapter 7. Graphs

7.1 Representing a graph

Input is usually n nodes and m edges "a b" (often 1-indexed). Use an **adjacency list**: `vector<vector<int>> adj(n + 1);`. For weighted graphs store pairs (neighbor, weight). An adjacency matrix is only for n ≤ ~2000 (Floyd-Warshall).

```
int n, m; cin >> n >> m;
vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; i++) {
    int a, b; cin >> a >> b;
    adj[a].push_back(b);
    adj[b].push_back(a);          // remove this line for a DIRECTED graph
}
```

Which algorithm?

Question	Algorithm	Time
Reachability, components	DFS / BFS / DSU	O(n + m)
Shortest path, no weights	BFS	O(n + m)
Shortest path, weights 0/1	0-1 BFS	O(n + m)
Shortest path, non-negative weights	Dijkstra	O(m log n)
Negative weights, negative cycle	Bellman-Ford	O(n·m)
All pairs, n ≤ 500	Floyd-Warshall	O(n^3)
Order with dependencies	Topological sort	O(n + m)
Cheapest way to connect everything	MST (Kruskal)	O(m log m)
Mutual reachability in a directed graph	SCC	O(n + m)
Critical edges / nodes	Bridges / articulation points	O(n + m)
Max flow / min cut / matching	Dinic / Hopcroft-Karp	see appendix

The complete, commented code for every algorithm is in **Appendix A**. This chapter shows how each one is used on a real problem.

7.2 BFS: shortest path in an unweighted graph

Problem (CSES 1667 Message Route): n computers, m connections. Find the minimum number of computers on a route from 1 to n and print the route, or "IMPOSSIBLE".

Approach: BFS from 1 gives the fewest edges. Store each node's parent, then walk back from n.

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }
    vector<int> dist(n + 1, -1), par(n + 1, 0);
    queue<int> q;
    dist[1] = 0; q.push(1);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; par[v]
= u; q.push(v); }
    }
    if (dist[n] == -1) { cout << "IMPOSSIBLE\n"; return 0; }
    vector<int> path;
    for (int v = n; v != 1; v = par[v]) path.push_back(v);
    path.push_back(1);
    reverse(path.begin(), path.end());
    cout << path.size() << "\n";
    for (int i = 0; i < (int)path.size(); i++) cout << path[i] << (i + 1 ==
(int)path.size() ? '\n' : ' ');
}
```

Grid BFS (CSES 1193 Labyrinth): cells are nodes, the 4 neighbors are edges. Store the direction used to enter each cell to rebuild the path as a string of L/R/U/D. **Multi-source BFS (CSES 1194 Monsters):** push all monsters at distance 0 to get the time each cell becomes dangerous, then BFS the player and only enter cells the player reaches strictly earlier.

7.3 DFS / connected components

Problem (CSES 1666 Building Roads): n cities, m roads. Add the minimum number of roads so that all cities are connected, and print them.

Approach: count connected components k; answer is $k - 1$ roads, connecting one representative of each component to the next.

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }
    vector<bool> vis(n + 1, false);
    vector<int> reps;                          // one node from each component
    for (int s = 1; s <= n; s++) {
        if (vis[s]) continue;
        reps.push_back(s);
        stack<int> st; st.push(s); vis[s] = true;   // iterative DFS
        while (!st.empty()) {
```

```
            int u = st.top(); st.pop();
            for (int v : adj[u]) if (!vis[v]) { vis[v] = true; st.push(v); }
        }
    }
    cout << reps.size() - 1 << "\n";
    for (int i = 1; i < (int)reps.size(); i++) cout << reps[i - 1] << ' ' <<
reps[i] << "\n";
}
```

7.4 Bipartite check (2-coloring)

Problem (CSES 1668 Building Teams): split pupils into two teams so that no two friends are in the same team. Print the team of each pupil (1 or 2) or "IMPOSSIBLE".

Approach: BFS each component, color neighbors with the opposite color. A conflict means an odd cycle → impossible.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
adj[b].push_back(a); }
    vector<int> col(n + 1, 0);                  // 0 = not colored, 1 or 2
    for (int s = 1; s <= n; s++) {
        if (col[s]) continue;
        col[s] = 1;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (!col[v]) { col[v] = 3 - col[u]; q.push(v); }
                else if (col[v] == col[u]) { cout << "IMPOSSIBLE\n"; return 0; }
            }
        }
    }
    for (int i = 1; i <= n; i++) cout << col[i] << (i == n ? '\n' : ' ');
}
```

7.5 Dijkstra

Problem (CSES 1671 Shortest Routes I): directed weighted graph, print the shortest distance from city 1 to every city.

Approach: priority queue of (distance, node). Pop the closest unfinished node, relax its edges. Skip outdated queue entries. Use long long distances.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
```

```
    int n, m; cin >> n >> m;
    vector<vector<pair<int,ll>>> adj(n + 1);
    for (int i = 0; i < m; i++) { int a, b; ll c; cin >> a >> b >> c;
adj[a].push_back({b, c}); }
    vector<ll> dist(n + 1, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    dist[1] = 0; pq.push({0, 1});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;              // outdated entry
        for (auto [v, w] : adj[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    for (int i = 1; i <= n; i++) cout << dist[i] << (i == n ? '\n' : ' ');
}
```

Dijkstra on an extended state. CSES 1195 Flight Discount: you may halve ONE flight's price. State = (city, used discount 0/1); edges: (u,0)→(v,0) cost c, (u,0)→(v,1) cost c/2, (u,1)→(v,1) cost c. Same trick for "at most k free edges", "fuel level", "time modulo k".

k shortest routes (CSES 1196): let each node be popped up to k times; the k-th pop of node n gives the k-th shortest route.

7.6 Floyd-Warshall

Problem (CSES 1672 Shortest Routes II): n ≤ 500 cities, undirected roads, q queries "shortest distance between a and b" (−1 if unreachable).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, q; cin >> n >> m >> q;
    const ll INF = 1e18;
    vector<vector<ll>> d(n + 1, vector<ll>(n + 1, INF));
    for (int i = 1; i <= n; i++) d[i][i] = 0;
    for (int i = 0; i < m; i++) {
        int a, b; ll c; cin >> a >> b >> c;
        d[a][b] = min(d[a][b], c); d[b][a] = min(d[b][a], c);   // keep the
cheapest road
    }
    for (int k = 1; k <= n; k++)                        // k outermost!
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (d[i][k] < INF && d[k][j] < INF) d[i][j] = min(d[i][j], d[i]
[k] + d[k][j]);
    while (q--) {
        int a, b; cin >> a >> b;
        cout << (d[a][b] == INF ? -1 : d[a][b]) << "\n";
    }
}
```

7.7 Bellman-Ford: negative cycles

Problem (CSES 1197 Cycle Finding): directed graph with possibly negative weights. Is there a negative cycle? If yes, print one.

Approach: run n rounds of relaxing all edges (start with all distances 0, so every node is a start). If round n still relaxes some node x, x is affected by a negative cycle. Walk parents n times from x to land inside the cycle, then follow parents until you return.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n, m; cin >> n >> m;
    vector<array<ll,3>> e(m);
    for (auto& [a, b, c] : e) cin >> a >> b >> c;
    vector<ll> dist(n + 1, 0);
    vector<int> par(n + 1, -1);
    int x = -1;
    for (int it = 0; it < n; it++) {
        x = -1;
        for (auto [a, b, c] : e)
            if (dist[a] + c < dist[b]) { dist[b] = dist[a] + c; par[b] = a; x =
b; }
    }
    if (x == -1) { cout << "NO\n"; return 0; }
    for (int i = 0; i < n; i++) x = par[x];     // now x is surely on the cycle
    vector<int> cyc = {x};
    for (int v = par[x]; v != x; v = par[v]) cyc.push_back(v);
    cyc.push_back(x);
    reverse(cyc.begin(), cyc.end());
    cout << "YES\n";
    for (int i = 0; i < (int)cyc.size(); i++) cout << cyc[i] << (i + 1 ==
(int)cyc.size() ? '\n' : ' ');
}
```

(Any negative cycle is accepted; the printed one may differ from the sample.)

7.8 Topological sort and cycle detection

Problem (CSES 1679 Course Schedule): n courses with m requirements "a before b". Print a valid order or "IMPOSSIBLE".

Approach (Kahn): repeatedly take a course with no remaining prerequisites (in-degree 0). If fewer than n courses are taken, there is a cycle.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].push_back(b);
indeg[b]++; }
```

```
    queue<int> q;
    for (int v = 1; v <= n; v++) if (!indeg[v]) q.push(v);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() < n) { cout << "IMPOSSIBLE\n"; return 0; }
    for (int i = 0; i < n; i++) cout << order[i] << (i + 1 == n ? '\n' : ' ');
}
```

Finding a cycle with DFS colors (CSES 1678 Round Trip II): color 0 = unvisited, 1 = on the current DFS path, 2 = finished. An edge to a color-1 node closes a cycle; rebuild it with parents. For undirected graphs (CSES 1669 Round Trip) an edge to a visited node that is not the parent closes a cycle.

7.9 Minimum spanning tree and DSU

Problem (CSES 1675 Road Reparation): choose roads to repair so that all cities are connected with minimum total cost, or "IMPOSSIBLE".

Approach (Kruskal): sort roads by cost; take a road if it connects two different components (DSU).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { while (x != p[x]) x = p[x] = p[p[x]]; return x; }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
int main() {
    int n, m; cin >> n >> m;
    vector<array<ll,3>> e(m);                  // {cost, a, b}
    for (auto& [c, a, b] : e) cin >> a >> b >> c;
    sort(e.begin(), e.end());
    DSU d(n);
    ll total = 0; int used = 0;
    for (auto [c, a, b] : e) if (d.unite(a, b)) { total += c; used++; }
    if (used == n - 1) cout << total << "\n";
    else cout << "IMPOSSIBLE\n";
}
```

7.10 Strongly connected components

Problem (CSES 1683 Planets and Kingdoms): directed graph; a kingdom is a maximal set where every planet can reach every other. Print the number of kingdoms and a kingdom label for each planet.

Approach (Kosaraju): DFS 1 on the graph records the finish order; DFS 2 on the reversed graph in reverse finish order — each DFS tree is one SCC. Iterative versions avoid stack overflow for n = 10^5.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> g(n + 1), rg(n + 1);
    for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; g[a].push_back(b);
rg[b].push_back(a); }
    // DFS 1 (iterative) -> finish order
    vector<int> order, it(n + 1, 0);
    vector<bool> vis(n + 1, false);
    for (int s = 1; s <= n; s++) {
        if (vis[s]) continue;
        vector<int> st = {s}; vis[s] = true;
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                int v = g[u][it[u]++];
                if (!vis[v]) { vis[v] = true; st.push_back(v); }
            } else { order.push_back(u); st.pop_back(); }   // u finished
        }
    }
    // DFS 2 on reversed graph, in reverse finish order
    vector<int> comp(n + 1, 0);
    int k = 0;
    for (int i = n - 1; i >= 0; i--) {
        int s = order[i];
        if (comp[s]) continue;
        k++;
        vector<int> st = {s}; comp[s] = k;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : rg[u]) if (!comp[v]) { comp[v] = k; st.push_back(v); }
        }
    }
    cout << k << "\n";
    for (int i = 1; i <= n; i++) cout << comp[i] << (i == n ? '\n' : ' ');
}
```

After SCCs: build the **condensation DAG** (one node per SCC) and do DP on it, e.g. maximum coins collected on a path (CSES 1686 Coin Collector). 2-SAT (CSES 1684 Giant Pizza) is also solved with SCC.

7.11 Max flow

Problem (CSES 1694 Download Speed): n computers, m directed connections with capacities; maximum download speed from computer 1 to n.

Approach: it is exactly max flow. Use Dinic (Appendix A, section 17). Min cut (CSES 1695 Police Chase) = after the flow, edges from the side reachable from the source to the other side. Max matching (CSES 1696 School Dance) = flow from a source to boys, boys to girls, girls to a sink, all capacity 1 (or Hopcroft-Karp).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Dinic {
    struct E { int to; ll cap; };
    int n; vector<E> e; vector<vector<int>> g; vector<int> level, it;
    Dinic(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll c) { g[u].push_back(e.size()); e.push_back({v,
c}); g[v].push_back(e.size()); e.push_back({u, 0}); }
    bool bfs(int s, int t) {
        level.assign(n, -1); queue<int> q; level[s] = 0; q.push(s);
        while (!q.empty()) { int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && level[e[id].to] < 0) {
level[e[id].to] = level[u] + 1; q.push(e[id].to); } }
        return level[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i], v = e[id].to;
            if (e[id].cap > 0 && level[v] == level[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) { it.assign(n, 0); while (ll f = dfs(s, t, LLONG_MAX))
flow += f; }
        return flow;
    }
};
int main() {
    int n, m; cin >> n >> m;
    Dinic d(n + 1);
    for (int i = 0; i < m; i++) { int a, b; ll c; cin >> a >> b >> c;
d.addEdge(a, b, c); }
    cout << d.maxflow(1, n) << "\n";
}
```

7.12 Graph modelling tips

- **States as nodes:** (cell, keys collected), (city, discount used), (position, time mod k). Then run BFS/Dijkstra on the state graph.
- **Reverse the graph** to compute "distance from every node TO the target" with one run.
- **Super source:** add node 0 connected to all start nodes with cost 0 → multi-source in one run.
- **Binary search + graph:** "minimum possible maximum edge on a path" = binary search the limit and check connectivity, or build the MST.
- **Edges with the same cost:** BFS. Costs 0/1: 0-1 BFS with a deque.
- **Is the graph a tree?** connected and $m = n - 1$.

Chapter 8. Trees

A tree has n nodes and n − 1 edges, is connected, and has exactly one path between any two nodes. Root it (usually at 1) and many problems become "combine answers of the children".

8.1 Subtree sizes

Problem (CSES 1674 Subordinates): employee 1 is the boss; for employees 2..n you are given their direct boss. Print how many subordinates each employee has.

Approach: subtree size − 1. Because boss[i] < i is not guaranteed in general problems, use a DFS/BFS order and accumulate from the bottom. Here the input gives parents directly.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<vector<int>> ch(n + 1);
    for (int i = 2; i <= n; i++) { int b; cin >> b; ch[b].push_back(i); }
    // BFS order from the root, then process in reverse (children before parents)
    vector<int> order = {1};
    for (int i = 0; i < (int)order.size(); i++)
        for (int c : ch[order[i]]) order.push_back(c);
    vector<int> sz(n + 1, 1);
    for (int i = n - 1; i >= 0; i--)
        for (int c : ch[order[i]]) sz[order[i]] += sz[c];
    for (int i = 1; i <= n; i++) cout << sz[i] - 1 << (i == n ? '\n' : ' ');
}
```

8.2 Tree diameter

Problem (CSES 1131 Tree Diameter): the maximum number of edges on a path between two nodes.

Approach: BFS from any node; the farthest node A is one end of a diameter. BFS from A; the farthest distance is the diameter. (For each node, its farthest distance = max(dist from A, dist from B) — CSES 1132 Tree Distances I.)

```
#include <bits/stdc++.h>
using namespace std;
int n;
vector<vector<int>> adj;
vector<int> bfs(int s) {
    vector<int> d(n + 1, -1);
    queue<int> q; d[s] = 0; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (d[v] == -1) { d[v] = d[u] + 1; q.push(v); }
```

```
    }
    return d;
}
int main() {
    cin >> n;
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b;
adj[a].push_back(b); adj[b].push_back(a); }
    vector<int> d1 = bfs(1);
    int A = max_element(d1.begin() + 1, d1.end()) - d1.begin();
    vector<int> dA = bfs(A);
    cout << *max_element(dA.begin() + 1, dA.end()) << "\n";
}
```

8.3 Binary lifting: k-th ancestor and LCA

up[j][v] = the 2^j-th ancestor of v. Build: up[j][v] = up[j−1][up[j−1][v]]. Then:

- **k-th ancestor:** for each set bit j of k, jump v = up[j][v]. O(log n).
- **LCA(a, b):** lift the deeper node to the same depth; if equal, done; else jump both up from the largest j while their ancestors differ; the answer is the parent.
- **Distance(a, b)** = depth[a] + depth[b] − 2·depth[LCA].

Problem (CSES 1688 Company Queries II): employee 1 is the boss; given the boss of 2..n, answer q queries "lowest common boss of a and b".

```
#include <bits/stdc++.h>
using namespace std;
const int LOG = 18;                    // 2^18 > 2*10^5
int up[LOG][200005], dep[200005];
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    up[0][1] = 1;                      // root points to itself
    vector<vector<int>> ch(n + 1);
    for (int i = 2; i <= n; i++) { cin >> up[0][i]; ch[up[0][i]].push_back(i); }
    // depths with BFS from the root
    vector<int> order = {1};
    for (int i = 0; i < (int)order.size(); i++)
        for (int c : ch[order[i]]) { dep[c] = dep[order[i]] + 1;
order.push_back(c); }
    for (int j = 1; j < LOG; j++)
        for (int v = 1; v <= n; v++) up[j][v] = up[j - 1][up[j - 1][v]];
    auto lift = [&](int v, int k) { for (int j = 0; j < LOG; j++) if (k >> j &
1) v = up[j][v]; return v; };
    while (q--) {
        int a, b; cin >> a >> b;
        if (dep[a] < dep[b]) swap(a, b);
        a = lift(a, dep[a] - dep[b]);              // same depth now
        if (a != b) {
            for (int j = LOG - 1; j >= 0; j--)
                if (up[j][a] != up[j][b]) { a = up[j][a]; b = up[j][b]; }
            a = up[0][a];
```

```
        }
        cout << a << "\n";
    }
}
```

8.4 Euler tour: subtree queries become range queries

Run a DFS and record tin[v] when entering and tout[v] when leaving. The subtree of v is exactly the positions tin[v] .. tout[v] in the order. A "sum of values in the subtree of v" query is then a range sum, handled by a Fenwick tree with point updates.

Problem (CSES 1137 Subtree Queries): node values; queries "1 s x" set the value of s to x, "2 s" print the sum of values in the subtree of s.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<ll> val(n + 1);
    for (int i = 1; i <= n; i++) cin >> val[i];
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b;
adj[a].push_back(b); adj[b].push_back(a); }
    // iterative DFS for tin / tout
    vector<int> tin(n + 1), tout(n + 1), par(n + 1, 0), it(n + 1, 0);
    int timer = 0;
    vector<int> st = {1}; tin[1] = ++timer;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)adj[u].size()) {
            int v = adj[u][it[u]++];
            if (v == par[u]) continue;
            par[v] = u; tin[v] = ++timer; st.push_back(v);
        } else { tout[u] = timer; st.pop_back(); }
    }
    vector<ll> fw(n + 1, 0);
    auto add = [&](int i, ll v) { for (; i <= n; i += i & -i) fw[i] += v; };
    auto sum = [&](int i) { ll s = 0; for (; i > 0; i -= i & -i) s += fw[i];
return s; };
    for (int v = 1; v <= n; v++) add(tin[v], val[v]);
    while (q--) {
        int type; cin >> type;
        if (type == 1) { int s; ll x; cin >> s >> x; add(tin[s], x - val[s]);
val[s] = x; }
        else { int s; cin >> s; cout << sum(tout[s]) - sum(tin[s] - 1) << "\n";
}
    }
}
```

Path queries (root to node): CSES 1138 Path Queries — updating v's value affects the root-path sum of every node in v's subtree: range add on [tin, tout], point query.

For queries on arbitrary paths a–b with updates use Heavy-Light Decomposition (Appendix A, section 23).

8.5 Rerooting technique

Compute something for EVERY root in O(n) total.

Problem (CSES 1133 Tree Distances II): for each node, the sum of distances to all other nodes.

Approach: 1. Root at 1. Compute sz[v] and ans[1] = Σ depth (one DFS). 2. Move the root from v to its child c: the sz[c] nodes in c's subtree get 1 closer, the other n − sz[c] get 1 farther. ans[c] = ans[v] − sz[c] + (n − sz[c]).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) { int a, b; cin >> a >> b;
adj[a].push_back(b); adj[b].push_back(a); }
    vector<int> order = {1}, par(n + 1, 0), dep(n + 1, 0);
    par[1] = -1;
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : adj[u]) if (v != par[u]) { par[v] = u; dep[v] = dep[u] + 1;
order.push_back(v); }
    }
    vector<ll> sz(n + 1, 1), ans(n + 1, 0);
    for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
    for (int v = 1; v <= n; v++) ans[1] += dep[v];
    for (int i = 1; i < n; i++) {                    // BFS order: parent before
child
        int c = order[i];
        ans[c] = ans[par[c]] - sz[c] + (n - sz[c]);
    }
    for (int v = 1; v <= n; v++) cout << ans[v] << (v == n ? '\n' : ' ');
}
```

8.6 Tree problem checklist

Problem asks	Technique
Something about every subtree	DFS post-order DP / small-to-large merging
Something for every root	Rerooting (two passes)
Distance / ancestor queries	Binary lifting LCA
Update a node, query a subtree	Euler tour + Fenwick / segment tree
Update and query arbitrary paths	Heavy-Light Decomposition
Count paths with a property (length k, ...)	Centroid decomposition
Distinct colors in each subtree (CSES 1139)	Small-to-large merging of sets
Maximum matching / independent set	Tree DP with "node taken / not taken"

Chapter 9. Game Theory

9.1 Winning and losing positions

Two players, perfect information, optimal play: "who wins?". Every position is either

- **P-position (losing):** the player to move loses. A position with no moves is losing (normal rule: "who cannot move loses").
- **N-position (winning):** the player to move wins.

Rules: 1. A position is **winning** if SOME move leads to a losing position. 2. A position is **losing** if EVERY move leads to a winning position.

So small games are solved by DP over positions.

Problem (CSES 1729 Stick Game): a heap of n sticks; a move removes p sticks where p is from a given set P. The player who cannot move loses. For every heap size 1..n, print W or L.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, k; cin >> n >> k;
    vector<int> p(k);
    for (auto& x : p) cin >> x;
    vector<bool> win(n + 1, false);          // win[0] = false: no moves, you
lose
    for (int i = 1; i <= n; i++)
        for (int x : p)
            if (x <= i && !win[i - x]) { win[i] = true; break; }  // move to a
losing state
    for (int i = 1; i <= n; i++) cout << (win[i] ? 'W' : 'L');
    cout << "\n";
}
```

9.2 Nim

n heaps; a move removes any positive number of sticks from ONE heap. **The first player loses exactly when a1 XOR a2 XOR … XOR an = 0.**

Why: from XOR = 0 every move makes XOR ≠ 0; from XOR ≠ 0 you can always move back to XOR = 0 (let x = total XOR, pick a heap with ai XOR x < ai, reduce it to ai XOR x).

Problem (CSES 1730 Nim Game I): t tests; print "first" or "second".

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int t; cin >> t;
    while (t--) {
```

```
        int n; cin >> n;
        int x = 0;
        for (int i = 0; i < n; i++) { int a; cin >> a; x ^= a; }
        cout << (x ? "first" : "second") << "\n";
    }
}
```

Nim variants with formulas

Game	First player LOSES when
Take 1..k from one heap (Bash)	(a1 mod (k+1)) XOR ... = 0; one heap: n mod (k+1) = 0
Nim Game II (CSES 1098): take 1, 2 or 3	XOR of (ai mod 4) = 0
Misère Nim (last mover LOSES)	all heaps ≤ 1: number of 1-heaps is odd; otherwise XOR = 0
Staircase Nim (CSES 1099 Stair Game): move balls from stair i to i−1	XOR of the balls on stairs 2, 4, 6, ... (1-indexed, stair 1 is the floor) = 0
Moore's Nim-k: remove from up to k heaps at once	each bit: (count of heaps with that bit) mod (k+1) = 0
Wythoff (two heaps; take from one, or the same from both)	a ≤ b and a = floor((b−a)·φ), φ = (1+√5)/2
Poker Nim (you may also add stones)	same as Nim

Stair game detail (CSES 1099): stairs 1..n, ball moves from stair k to k−1, stair 1 is the floor where balls stop. Only stairs 2, 4, 6, ... (even positions in 1-indexing) matter.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        int x = 0;
        for (int i = 1; i <= n; i++) { int p; cin >> p; if (i % 2 == 0) x ^= p;
}
        cout << (x ? "first" : "second") << "\n";
    }
}
```

Why: balls on odd stairs are "dead" — whatever the opponent moves from an odd stair to an even one, you move the same balls down again to the next odd stair.

9.3 Sprague-Grundy theorem

Every impartial game position has a **grundy number** g:

- g(position) = **mex** of the grundy numbers of all positions you can move to. mex(S) = smallest non-negative integer not in S. A position with no moves has g = 0.

- The position is losing exactly when g = 0.
- **Sum of independent games** (each turn move in exactly one): g = XOR of the parts' grundy numbers.
- A move that splits a game into two independent games leads to g(left) XOR g(right).

Recipe: compute g for ONE heap/component (DP), XOR over all heaps, first player wins iff the XOR ≠ 0.

Problem (CSES 2207 Grundy's Game): one heap of n coins; a move splits a heap into two NON-EMPTY heaps of DIFFERENT sizes. Who wins? (n ≤ 10^6, t ≤ 10^5 tests.)

Approach: g(n) = mex over a < n − a of g(a) XOR g(n − a). Computing up to 10^6 is too slow, but it is known (and easy to observe by printing) that g(n) > 0 for every n > 1222 except a few small values — so compute grundy up to ~2000 and answer "first" for anything larger.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    const int LIM = 2000;
    vector<int> g(LIM + 1, 0);
    for (int n = 3; n <= LIM; n++) {
        vector<bool> seen(LIM, false);
        for (int a = 1; 2 * a < n; a++) seen[g[a] ^ g[n - a]] = true;  // split
into a and n-a, a < n-a
        int m = 0;
        while (seen[m]) m++;
        g[n] = m;
    }
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        bool first = (n > LIM) ? true : g[n] != 0;
        cout << (first ? "first" : "second") << "\n";
    }
}
```

General grundy template

```
// g[n] for a single heap where allowed moves remove an amount from set S
vector<int> grundy(int maxN, const vector<int>& S) {
    vector<int> g(maxN + 1, 0);
    for (int n = 1; n <= maxN; n++) {
        vector<bool> seen(S.size() + 2, false);     // mex <= number of moves
        for (int s : S) if (s <= n && g[n - s] < (int)seen.size()) seen[g[n -
s]] = true;
        int m = 0;
        while (seen[m]) m++;
        g[n] = m;
    }
    return g;
}
```

One heap of n, moves	g(n)
Take any amount (Nim)	n
Take 1..k	n mod (k+1)
Take any amount OR split into two non-empty heaps (Lasker)	n mod 4 = 1, 2: n; 3: n+1; 0: n−1
Remove 1 or 2 adjacent pins from a row (Kayles)	DP; period 12 from about n = 71
Anything else	Print g for n = 0..100, find the period / pattern

9.4 Games on graphs

- **DAG:** grundy of each node by memoized DFS. Several tokens moving independently: XOR their grundy numbers.
- **Graph with cycles (Draw possible):** retrograde BFS from terminal nodes: a node with an edge to a LOSE node is WIN; a node whose every edge goes to WIN nodes is LOSE (keep a counter of remaining out-edges); never-labelled nodes are DRAW.
- **Trees (Green Hackenbush):** a move cuts an edge and removes everything disconnected from the root; g(v) = XOR over children c of (g(c) + 1).

```
// res[u]: 1 = win, -1 = lose, 0 = draw, for the player to move at u
vector<int> retrograde(int n, const vector<vector<int>>& adj) {
    vector<vector<int>> radj(n);
    vector<int> deg(n), res(n, 0);
    for (int u = 0; u < n; u++) { deg[u] = adj[u].size(); for (int v : adj[u])
radj[v].push_back(u); }
    queue<int> q;
    for (int u = 0; u < n; u++) if (!deg[u]) { res[u] = -1; q.push(u); }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : radj[v]) {
            if (res[u]) continue;
            if (res[v] == -1) { res[u] = 1; q.push(u); }                // can
move to a losing node
            else if (--deg[u] == 0) { res[u] = -1; q.push(u); }         // all
moves go to winning nodes
        }
    }
    return res;
}
```

9.5 Score games (minimax)

When players collect points, store dp = best (my score − opponent's score) for the player to move. The opponent's best is subtracted. Example: the Removal Game in section 4.8: dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]). With n ≤ 20 items taken in any order, use dp[mask] the same way.

9.6 Game theory checklist

1. Normal rule (who cannot move loses) or misère (who moves last loses)? Draws possible?
2. Impartial (same moves for both)? → win/lose DP or grundy.
3. Independent parts? → XOR of grundy numbers.
4. Known game? Compare with the tables above.
5. Huge n? → brute force small n, print W/L or grundy values, find the pattern (period, mod, powers of two, Fibonacci).
6. Symmetry: the second player can often mirror; the first player can often take the center and then mirror.

Chapter 10. Geometry, Bit Tricks and Other Topics

10.1 Geometry basics with integers

Use `long long` coordinates whenever the input is integer — no precision problems.

Cross product of vectors (x1, y1) and (x2, y2): x1·y2 − x2·y1. For points A, B, C, cross(B − A, C − A) is

- 0: C is to the LEFT of the line A→B (counter-clockwise turn)
- < 0: C is to the RIGHT (clockwise turn)
- = 0: A, B, C are collinear

Problem (CSES 2189 Point Location Test): for each test, is point p3 to the LEFT, RIGHT or TOUCH the line p1→p2?

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct P { ll x, y; };
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll orient(P a, P b, P c) { return cross(b - a, c - a); }   // sign tells the
turn
int main() {
    int t; cin >> t;
    while (t--) {
        P a, b, c; cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y;
        ll o = orient(a, b, c);
        cout << (o > 0 ? "LEFT" : o < 0 ? "RIGHT" : "TOUCH") << "\n";
    }
}
```

Do segments AB and CD intersect? (CSES 2190) Yes if C and D are on different sides of AB AND A and B are on different sides of CD. If some orientation is 0, check whether that point lies on the other segment (inside its bounding box).

```
bool onSegment(P a, P b, P p) {          // p is collinear with a, b
    return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) && min(a.y, b.y) <= p.y
&& p.y <= max(a.y, b.y);
}
int sgn(ll v) { return (v > 0) - (v < 0); }
bool intersect(P a, P b, P c, P d) {
    ll d1 = orient(a, b, c), d2 = orient(a, b, d), d3 = orient(c, d, a), d4 =
orient(c, d, b);
    if (d1 == 0 && onSegment(a, b, c)) return true;
    if (d2 == 0 && onSegment(a, b, d)) return true;
    if (d3 == 0 && onSegment(c, d, a)) return true;
    if (d4 == 0 && onSegment(c, d, b)) return true;
```

```
    return sgn(d1) * sgn(d2) < 0 && sgn(d3) * sgn(d4) < 0;
}
```

Polygon area (shoelace formula, CSES 2191): 2·Area = |Σ cross(p[i], p[i+1])|. Print the doubled area to stay in integers.

Pick's theorem (CSES 2193 Polygon Lattice Points): Area = I + B/2 − 1, where B = lattice points on the boundary = Σ gcd(|dx|, |dy|) over edges, I = interior lattice points.

10.2 Convex hull (Andrew's monotone chain)

Problem (CSES 2195 Convex Hull): print the points on the convex hull (including collinear points on the boundary).

Approach: sort points by (x, y). Build the lower hull left to right and the upper hull right to left, popping while the last turn is clockwise (cross < 0 keeps collinear points; use ≤ 0 to drop them).

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct P { ll x, y; bool operator<(const P& o) const { return x < o.x || (x == o.x && y < o.y); } };
ll cross(P o, P a, P b) { return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x); }
int main() {
    int n; cin >> n;
    vector<P> p(n);
    for (auto& q : p) cin >> q.x >> q.y;
    sort(p.begin(), p.end());
    vector<P> hull;
    for (int phase = 0; phase < 2; phase++) {            // 0 = lower hull, 1 = upper hull
        int start = hull.size();
        for (auto& q : p) {
            while ((int)hull.size() >= start + 2 && cross(hull[hull.size() - 2], hull.back(), q) < 0)
                hull.pop_back();                         // right turn -> not on the hull
            hull.push_back(q);
        }
        hull.pop_back();                                 // last point repeats in the other half
        reverse(p.begin(), p.end());
    }
    // with collinear points kept, remove duplicates that can appear when all points are collinear
    sort(hull.begin(), hull.end());
    hull.erase(unique(hull.begin(), hull.end(), [](P a, P b){ return a.x == b.x && a.y == b.y; }), hull.end());
    cout << hull.size() << "\n";
    // print in counter-clockwise order starting from the lowest-left point
    P c = hull[0];
```

```
    vector<P> rest(hull.begin() + 1, hull.end());
    sort(rest.begin(), rest.end(), [&](P a, P b) {
        ll cr = cross(c, a, b);
        if (cr != 0) return cr > 0;
        return (a.x - c.x) * (a.x - c.x) + (a.y - c.y) * (a.y - c.y) < (b.x -
c.x) * (b.x - c.x) + (b.y - c.y) * (b.y - c.y);
    });
    cout << c.x << ' ' << c.y << "\n";
    for (auto& q : rest) cout << q.x << ' ' << q.y << "\n";
}
```

(Any order is accepted by CSES; the printing step above is only for readable output.)

10.3 Bit manipulation tricks

Expression	Meaning
`x & 1`	x is odd
`x >> i & 1`	bit i of x
`x & (x − 1)`	x without its lowest set bit; `== 0` means x is a power of two
`x & −x`	lowest set bit value
`__builtin_popcountll(x)`	number of set bits
`__builtin_clzll(x)`	leading zeros; `63 − clzll(x)` = floor(log2 x)
`__builtin_ctzll(x)`	trailing zeros = index of lowest set bit
`a ^ b ^ b == a`	XOR cancels: find the unique element among pairs
`a + b == (a ^ b) + 2·(a & b)`	relation between sum, XOR and AND

XOR facts used in problems

- XOR of 1..n: n%4 = 0 → n, 1 → 1, 2 → n+1, 3 → 0.
- Prefix XOR: xor(l..r) = px[r] ^ px[l−1] (CSES 1650 Range Xor Queries).
- Max XOR of two numbers: binary trie (section 6.5).
- **Contribution per bit:** for "sum of XOR / AND / OR over all pairs", handle each bit independently: count how many numbers have bit b set (c) → pairs with XOR bit set = c·(n − c), contributing c·(n − c)·2^b.

```
// Sum of (a[i] XOR a[j]) over all pairs i < j, by bit contribution
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; cin >> n;
    vector<ll> a(n);
    for (auto& x : a) cin >> x;
    ll ans = 0;
    for (int b = 0; b < 31; b++) {
```

```
        ll c = 0;
        for (ll x : a) c += x >> b & 1;
        ans += c * (n - c) * (1LL << b);     // pairs with different bit b
    }
    cout << ans << "\n";
}
```

Gray code (CSES 2205): the i-th code is i ^ (i >> 1); consecutive codes differ in exactly one bit.

10.4 Constructive problems

"Print any array/permutation/graph that satisfies ...". No standard algorithm; tips:

- Solve tiny cases by hand (n = 1..6) and look for a pattern; often odd and even n differ.
- Find a NECESSARY condition first (parity, sum, counts); then try to show it is also sufficient by building it.
- Try simple shapes: sorted order, alternating, 1, n, 2, n−1, ...; a star or a path for graphs.
- Write a brute force that tries all permutations for small n and prints the valid ones.

Example (CSES 1070 Permutations): a permutation of 1..n where adjacent values never differ by 1. n = 1 works; n = 2, 3 impossible; otherwise print all even numbers, then all odd numbers.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    if (n == 1) { cout << 1 << "\n"; return 0; }
    if (n <= 3) { cout << "NO SOLUTION\n"; return 0; }
    vector<int> res;
    for (int i = 2; i <= n; i += 2) res.push_back(i);   // evens first
    for (int i = 1; i <= n; i += 2) res.push_back(i);   // then odds
    for (int i = 0; i < n; i++) cout << res[i] << (i + 1 == n ? '\n' : ' ');
}
```

10.5 Interactive problems

- Flush after every output: `cout << "? " << x << endl;` (endl flushes) or `cout.flush()`.
- Read the judge's reply before asking the next question; stop immediately on −1.
- Usually binary search: find a hidden number in [1, n] with about log2(n) questions.

```
int lo = 1, hi = n;
while (lo < hi) {
    int mid = (lo + hi) / 2;
```

```
    cout << "? " << mid << endl;          // ask: is the number > mid ?
    string ans; cin >> ans;
    if (ans == "YES") lo = mid + 1; else hi = mid;
}
cout << "! " << lo << endl;
```

10.6 Probability and expected value

- Linearity of expectation: E[X + Y] = E[X] + E[Y] even if X and Y are dependent. Split the quantity into indicator variables ("is item i chosen?") and add their probabilities.
- Expected number of trials until success with probability p: 1/p.
- Modular probability: a fraction P/Q mod M is P · Q^(M−2) mod M.
- DP over states with probabilities: dp[state] = Σ p(move) · dp[next].

Example (CSES 1725 Dice Probability): probability that the sum of n dice is in [a, b]: dp[i][s] = probability of sum s after i dice; dp[i][s] = Σ over face f of dp[i−1][s−f] / 6.

Chapter 11. Problem Pattern Index (read this during the contest)

When you are stuck, match the problem's words or limits to a technique.

11.1 From keywords to techniques

The statement says…	Think about
"minimum number of operations", grid / graph, equal costs	BFS
"minimum cost path", positive weights	Dijkstra
"number of ways", "modulo 10^9+7"	DP (or combinatorics nCr)
"maximum/minimum of the maximum/minimum"	Binary search on the answer
"subarray", "contiguous", "window"	Prefix sums, two pointers, sliding window, Kadane
"subsequence"	DP (LIS, LCS), greedy with a stack
"intervals", "meetings", "segments"	Sort by end/start, sweep line, greedy
"queries on a range", "update"	Fenwick / segment tree / sparse table
"queries offline", no updates, distinct values	Mo's algorithm, sort queries by right end
"n ≤ 20"	Bitmask DP, subsets
"n ≤ 40"	Meet in the middle
"n ≤ 500"	O(n^3): Floyd, interval DP
"number up to 10^18"	Math, binary exponentiation, digit DP, matrix power
"k-th smallest"	Binary search on value, ordered set, heap
"connected", "groups", "merge"	DSU
"dependencies", "order of tasks"	Topological sort
"tree", "root", "ancestor"	DFS, LCA, tree DP, Euler tour
"both players play optimally"	Game theory: win/lose DP, Nim, grundy
"string occurs", "pattern"	KMP / Z / hashing
"palindrome"	Expand around center, Manacher, interval DP
"XOR"	Bit-by-bit thinking, prefix XOR, binary trie, linear basis
"GCD", "divisible", "prime"	Sieve, factorization, divisor counting
"permutation" with a property	Constructive: try small n, find a pattern
"expected value", "probability"	Linearity of expectation, probability DP
"matching", "assign", "pairs" between two groups	Bipartite matching / flow / Hungarian
"cut", "separate", "disconnect"	Min cut, bridges, articulation points

The statement says…	Think about
"cycle" in a directed graph	DFS colors, topological sort, SCC

11.2 Standard tricks worth remembering

1. **Sort first.** Many O(n^2) ideas become O(n log n) after sorting.
2. **Fix one thing, optimize the rest.** Fix the left end / the minimum / the middle element and count or optimize over the rest.
3. **Count the complement.** Total − (bad cases) is often easier than counting good cases.
4. **Contribution technique.** Instead of summing over all subarrays/pairs, ask "how many times is a[i] counted?" — a[i] is in (i + 1)·(n − i) subarrays (0-indexed).
5. **Reverse the process.** Deletions → additions in reverse (DSU), "last operation wins" → process from the end.
6. **Think in terms of the answer.** If checking a candidate answer is easy, binary search it.
7. **Look at the parity or the invariant.** Some operations never change the sum mod 2, the XOR, or the number of inversions mod 2.
8. **Small values, big counts.** If values ≤ 10^6, use counting arrays instead of sorting.
9. **Two passes.** Compute something from the left and from the right (prefix max / suffix max), then combine.
10. **Pigeonhole.** With n+1 prefix sums mod n, two are equal → a subarray with sum divisible by n always exists.

11.3 Kadane's algorithm (maximum subarray sum)

Problem (CSES 1643 Maximum Subarray Sum): maximum sum of a non-empty contiguous subarray.

Approach: cur = best sum of a subarray ending here = max(a[i], cur + a[i]). Answer = max over all cur.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    int n; cin >> n;
    ll cur = 0, best = LLONG_MIN;
    for (int i = 0; i < n; i++) {
        ll x; cin >> x;
        cur = max(x, cur + x);     // extend the previous subarray or start
fresh
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

11.4 Final contest-day checklist

- ☐ Template typed and tested; fast I/O on.
- ☐ Read ALL problems in the first minutes; sort by difficulty (look at the scoreboard).
- ☐ Before submitting: long long? array sizes? multiple test cases reset? output format?
- ☐ Test the sample AND the edge cases: n = 1, all equal, maximum values, negative numbers.
- ☐ On Wrong Answer: re-read the statement, then stress-test with a brute force.
- ☐ On Time Limit: check complexity against the table in 1.2; remove `endl`; avoid `map` in hot loops.
- ☐ On Runtime Error: out of bounds, division by zero, recursion too deep, uninitialized pointer.
- ☐ Team: one person codes, one reads the next problem, one prepares tests.

Appendix A. Complete Graph Template (24 algorithms)

Every algorithm from the graph template file, ready to copy. Each section starts with WHAT IT DOES and a COMMON PROBLEM. The full single file (with a demo `main`) is the same code in order.

A.0 Header and table of contents

```
/*
 ================================================================================
   ICPC GRAPH ALGORITHMS TEMPLATE  (single file, C++17)
 ================================================================================
   HOW TO USE
   - Every algorithm is self-contained: copy only what you need.
   - Nodes are 0-indexed (0..n-1). If your problem is 1-indexed, just create
     the structure with size n+1 and use nodes 1..n.
   - main() at the bottom shows a small working example of every algorithm.
   - Compile:  g++ -std=c++17 -O2 graph_templates.cpp -o g

   TABLE OF CONTENTS                                          COMPLEXITY
   ---------------------------------------------------------------------------
    1. BFS (shortest path, unweighted) + path        O(V + E)
    2. Grid BFS (4 directions)                        O(R * C)
    3. DFS (iterative, no stack overflow)             O(V + E)
    4. 0-1 BFS (edge weights 0 or 1)                  O(V + E)
    5. DSU / Union-Find                               ~O(1) per op
    6. Topological Sort (Kahn) + cycle detection      O(V + E)
    7. Bipartite check (2-coloring)                   O(V + E)
    8. Dijkstra (non-negative weights) + path         O(E log V)
    9. Bellman-Ford (negative weights / cycles)       O(V * E)
   10. Floyd-Warshall (all pairs)                     O(V^3)
   11. Kruskal MST                                    O(E log E)
   12. Prim MST                                       O(E log V)
   13. LCA (binary lifting) + k-th ancestor + dist    O(N log N) build, O(log N)
query
   14. Bridges & Articulation Points                  O(V + E)
   15. SCC (Kosaraju) + Condensation DAG              O(V + E)
   16. 2-SAT                                          O(V + E)
   17. Max Flow (Dinic) + Min Cut                     O(V^2 E), very fast in
practice
   18. Min Cost Max Flow (SPFA)                       O(F * V * E)
   19. Bipartite Matching (Hopcroft-Karp)             O(E sqrt V)
   20. Hungarian (min-cost assignment)                O(n^2 m)
   21. Euler Path / Circuit (Hierholzer)              O(V + E)
   22. Tree Diameter                                  O(N)
   23. Heavy-Light Decomposition (path queries)       O(log^2 N) per query
   24. Centroid Decomposition (skeleton)              O(N log N)
 ================================================================================
*/
```

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;   // "infinity" for distances (safe to add small weights)
```

A.1 BFS — shortest path in an UNWEIGHTED graph

```cpp
// ============================================================================
// 1. BFS — shortest path in an UNWEIGHTED graph
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Finds the minimum number of edges from one node to all others.
// COMMON PROBLEM: Shortest number of moves/hops, e.g. 'minimum flights from city
A to B',
//                 'fewest knight moves to a square', multi-source 'distance to
nearest fire'.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// dist[v] = number of edges from src to v (-1 if unreachable).
// par[v]  = previous node on a shortest path (used to rebuild the path).
// Multi-source BFS: push ALL sources with dist 0 at the start.
// ============================================================================
struct BFS {
    vector<int> dist, par;
    void run(int src, const vector<vector<int>>& adj) {
        int n = adj.size();
        dist.assign(n, -1); par.assign(n, -1);
        queue<int> q;
        dist[src] = 0; q.push(src);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u])
                if (dist[v] == -1) {          // not visited yet
                    dist[v] = dist[u] + 1;
                    par[v] = u;
                    q.push(v);
                }
        }
    }
    // path from src to t (empty if unreachable)
    vector<int> path(int t) {
        vector<int> p;
        if (dist[t] == -1) return p;
        for (int v = t; v != -1; v = par[v]) p.push_back(v);
        reverse(p.begin(), p.end());
        return p;
    }
};
```

A.2 GRID BFS — shortest path on a 2D grid ('#' = wall, '.' = free)

```
// ============================================================================
// 2. GRID BFS — shortest path on a 2D grid ('#' = wall, '.' = free)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : BFS where cells of a 2D map are the nodes and moves are the
edges.
// COMMON PROBLEM: Maze: 'shortest path from S to E avoiding #',
//                 'count rooms' (flood fill), 'monsters + player escape' (multi-
source BFS).
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// For 8 directions use dx = {-1,-1,-1,0,0,1,1,1}, dy = {-1,0,1,-1,1,-1,0,1}.
// ============================================================================
const int dx[4] = {-1, 1, 0, 0};
const int dy[4] = {0, 0, -1, 1};

vector<vector<int>> gridBFS(const vector<string>& grid, int sr, int sc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int,int>> q;
    d[sr][sc] = 0; q.push({sr, sc});
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nr = r + dx[k], nc = c + dy[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue; // outside
            if (grid[nr][nc] == '#' || d[nr][nc] != -1) continue;  // wall /
seen
            d[nr][nc] = d[r][c] + 1;
            q.push({nr, nc});
        }
    }
    return d;
}
```

A.3 DFS — iterative (recursive DFS can overflow the stack for n ~ 1e6)

```
// ============================================================================
// 3. DFS — iterative (recursive DFS can overflow the stack for n ~ 1e6)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Visits everything reachable from a node; groups nodes into
components.
// COMMON PROBLEM: 'How many connected islands / friend groups?', 'is v reachable
from u?',
//                 'minimum roads to add so all cities connect' = components - 1.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Returns nodes in the order they were visited. Also counts components.
// ============================================================================
```

```cpp
vector<int> dfsOrder(int src, const vector<vector<int>>& adj, vector<bool>& vis)
{
    vector<int> order, st = {src};
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        if (vis[u]) continue;
        vis[u] = true;
        order.push_back(u);
        for (int v : adj[u]) if (!vis[v]) st.push_back(v);
    }
    return order;
}

int countComponents(const vector<vector<int>>& adj) {
    int n = adj.size(), comps = 0;
    vector<bool> vis(n, false);
    for (int i = 0; i < n; i++)
        if (!vis[i]) { dfsOrder(i, adj, vis); comps++; }
    return comps;
}
```

A.4 0-1 BFS — shortest path when every weight is 0 or 1

```cpp
// ================================================================================
// 4. 0-1 BFS — shortest path when every weight is 0 or 1
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Shortest path when edges cost only 0 or 1; faster than
Dijkstra.
// COMMON PROBLEM: 'Minimum walls to break to reach the exit' (free cell = 0,
wall = 1),
//                 'minimum edge reversals to go from 1 to n' (original = 0,
reversed = 1).
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Like Dijkstra but with a deque: weight-0 edges go to the FRONT,
// weight-1 edges go to the BACK. adj[u] = {(v, w)} with w in {0,1}.
// ================================================================================
vector<int> zeroOneBFS(int src, const vector<vector<pair<int,int>>>& adj) {
    int n = adj.size();
    vector<int> d(n, INT_MAX);
    deque<int> dq;
    d[src] = 0; dq.push_back(src);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u])
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                if (w == 0) dq.push_front(v); else dq.push_back(v);
            }
    }
    return d;
}
```

A.5 DSU (Disjoint Set Union / Union-Find)

```cpp
// ============================================================================
// 5. DSU (Disjoint Set Union / Union-Find)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Tracks which items are in the same group and merges groups
fast.
// COMMON PROBLEM: Online queries 'connect a b' / 'are a and b connected?',
//                 cycle detection in undirected graph, Kruskal MST, 'size of
largest group'.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// find(x)    : representative of x's set (path compression)
// unite(a,b) : merge sets, returns false if already in same set (=> cycle!)
// size(x)    : size of x's set
// ============================================================================
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (x != p[x]) x = p[x] = p[p[x]];   // path halving
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);           // union by size
        p[b] = a; sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return sz[find(x)]; }
};
```

A.6 TOPOLOGICAL SORT (Kahn's algorithm) — DIRECTED graph

```cpp
// ============================================================================
// 6. TOPOLOGICAL SORT (Kahn's algorithm) — DIRECTED graph
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Orders nodes of a DAG so every edge goes from earlier to
later.
// COMMON PROBLEM: 'Course schedule: order courses so prerequisites come first
(or IMPOSSIBLE)',
//                 task scheduling, DP on DAG (longest path / count paths),
dependency cycle check.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Repeatedly take a node with in-degree 0.
// If the result has fewer than n nodes => the graph has a CYCLE.
// For the lexicographically smallest order, replace queue with
// priority_queue<int, vector<int>, greater<int>>.
// ============================================================================
```

```cpp
vector<int> topoSort(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> indeg(n, 0), order;
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() != n) return {};      // cycle exists
    return order;
}
// Directed cycle check:   topoSort(adj).empty()  (for n > 0)
// Undirected cycle check: use DSU — if unite(u,v) returns false, there's a
cycle.
```

A.7 BIPARTITE CHECK — can we color nodes with 2 colors so every edge

```cpp
// ================================================================================
// 7. BIPARTITE CHECK — can we color nodes with 2 colors so every edge
//    connects different colors? (= graph has no odd cycle)
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Checks if nodes split into 2 groups with edges only across
groups.
// COMMON PROBLEM: 'Split students into 2 teams so no two friends share a team',
//                 'does the graph have an odd cycle?', first step before
bipartite matching.
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// color[v] in {0,1} on success.
// ================================================================================
bool isBipartite(const vector<vector<int>>& adj, vector<int>& color) {
    int n = adj.size();
    color.assign(n, -1);
    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;
        queue<int> q; q.push(s); color[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) { color[v] = color[u] ^ 1; q.push(v); }
                else if (color[v] == color[u]) return false;   // odd cycle
            }
        }
    }
    return true;
}
```

A.8 DIJKSTRA — single-source shortest path, NON-NEGATIVE weights

```cpp
// ================================================================================
// 8. DIJKSTRA — single-source shortest path, NON-NEGATIVE weights
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Shortest distance from one source with NON-NEGATIVE edge
weights.
// COMMON PROBLEM: 'Cheapest route from city 1 to every city',
//                 state-graph Dijkstra: (node, coupon_used) for 'one flight at
half price'.
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// adj[u] = {(v, w)}. dist[v] = INF if unreachable.
// ================================================================================
struct Dijkstra {
    vector<ll> dist;
    vector<int> par;
    void run(int src, const vector<vector<pair<int,ll>>>& adj) {
        int n = adj.size();
        dist.assign(n, INF); par.assign(n, -1);
        // min-heap of (distance, node)
        priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
        dist[src] = 0; pq.push({0, src});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d != dist[u]) continue;           // outdated entry, skip
            for (auto [v, w] : adj[u])
                if (dist[u] + w < dist[v]) {      // relax edge
                    dist[v] = dist[u] + w;
                    par[v] = u;
                    pq.push({dist[v], v});
                }
        }
    }
    vector<int> path(int t) {
        vector<int> p;
        if (dist[t] == INF) return p;
        for (int v = t; v != -1; v = par[v]) p.push_back(v);
        reverse(p.begin(), p.end());
        return p;
    }
};
```

A.9 BELLMAN-FORD — shortest paths with NEGATIVE weights

```cpp
// ================================================================================
// 9. BELLMAN-FORD — shortest paths with NEGATIVE weights
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Shortest paths with negative weights; detects negative cycles.
// COMMON PROBLEM: 'Is there a currency arbitrage loop?', 'can the score grow
```

```
infinitely?'
//                (negate weights, check -INF reaching n), systems of difference
constraints.
// ---------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Relax all edges n-1 times. If something still improves afterwards,
// there's a negative cycle. We then mark every node affected by a negative
// cycle with -INF (its distance can be made arbitrarily small).
// Returns true if NO negative cycle is reachable from src.
// ===========================================================================
struct Edge { int u, v; ll w; };

bool bellmanFord(int n, int src, const vector<Edge>& edges, vector<ll>& dist) {
    dist.assign(n, INF);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        bool changed = false;
        for (auto& e : edges)
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        if (!changed) break;                        // early exit
    }
    bool ok = true;
    // propagate -INF through everything reachable from a negative cycle
    for (int i = 0; i < n; i++)
        for (auto& e : edges) {
            if (dist[e.u] == INF) continue;
            if (dist[e.u] == -INF || dist[e.u] + e.w < dist[e.v]) {
                if (dist[e.v] != -INF) ok = false;
                dist[e.v] = -INF;
            }
        }
    return ok;
}
```

A.10 FLOYD-WARSHALL — all-pairs shortest paths (n <= ~500)

```
// ===========================================================================
// 10. FLOYD-WARSHALL — all-pairs shortest paths (n <= ~500)
// ---------------------------------------------------------------------------
// WHAT IT DOES  : Shortest distance between EVERY pair of nodes (small n).
// COMMON PROBLEM: 'Answer q queries: shortest path a->b' with n <= 500,
//                 transitive closure ('can i reach j?'), minimum-weight cycle in
a graph.
// ---------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// d[i][j] = edge weight (INF if no edge), d[i][i] = 0. Modified in place.
// Negative cycle exists if some d[i][i] < 0 afterwards.
// ===========================================================================
void floydWarshall(vector<vector<ll>>& d) {
    int n = d.size();
```

```
    for (int k = 0; k < n; k++)           // k = intermediate node (MUST be
outer)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

A.11 KRUSKAL — Minimum Spanning Tree

```
// ============================================================================
// 11. KRUSKAL — Minimum Spanning Tree
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Minimum Spanning Tree: connect all nodes with minimum total
edge cost.
// COMMON PROBLEM: 'Minimum cost to build roads so every city is connected',
//                 max spanning tree, 'minimize the largest edge on a path'
(bottleneck).
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Sort edges by weight, add an edge if it joins two different components.
// Returns total weight, or -1 if the graph is not connected.
// (For MAXIMUM spanning tree: sort in decreasing order.)
// ============================================================================
ll kruskal(int n, vector<Edge> edges, vector<Edge>* mstEdges = nullptr) {
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) { return
a.w < b.w; });
    DSU dsu(n);
    ll total = 0; int used = 0;
    for (auto& e : edges)
        if (dsu.unite(e.u, e.v)) {
            total += e.w; used++;
            if (mstEdges) mstEdges->push_back(e);
        }
    return used == n - 1 ? total : -1;
}
```

A.12 PRIM — Minimum Spanning Tree (good for dense graphs / adjacency lists)

```
// ============================================================================
// 12. PRIM — Minimum Spanning Tree (good for dense graphs / adjacency lists)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Same MST answer as Kruskal, but grows one tree from a start
node.
// COMMON PROBLEM: Dense graphs: 'connect all n points with minimum cable
length',
//                 use when the graph is given as an adjacency list or is
complete.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Grow the tree from node 0, always taking the cheapest edge leaving it.
```

```
// Returns total weight, or -1 if not connected.
// ============================================================================
ll prim(const vector<vector<pair<int,ll>>>& adj) {
    int n = adj.size(), cnt = 0;
    vector<bool> inTree(n, false);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    pq.push({0, 0});
    ll total = 0;
    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (inTree[u]) continue;
        inTree[u] = true; total += w; cnt++;
        for (auto [v, c] : adj[u]) if (!inTree[v]) pq.push({c, v});
    }
    return cnt == n ? total : -1;
}
```

A.13 LCA — Lowest Common Ancestor with BINARY LIFTING (on a TREE)

```
// ============================================================================
// 13. LCA — Lowest Common Ancestor with BINARY LIFTING (on a TREE)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Finds the deepest common ancestor of two tree nodes; fast
distance queries.
// COMMON PROBLEM: 'q queries: distance between nodes a and b in a tree',
//                 'k-th boss of an employee' (k-th ancestor), path max with
extra tables.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// up[j][v] = 2^j-th ancestor of v.
// lca(a,b)      : lowest common ancestor
// kth(v,k)      : k-th ancestor of v
// dist(a,b)     : weighted distance between a and b
// edgesBetween  : number of edges between a and b
// adj[u] = {(v, w)}. For an unweighted tree use w = 1.
// ============================================================================
struct LCA {
    int n, LOG;
    vector<vector<int>> up;
    vector<int> depth;
    vector<ll> distRoot;          // weighted distance from root
    LCA(const vector<vector<pair<int,ll>>>& adj, int root = 0) {
        n = adj.size(); LOG = 1;
        while ((1 << LOG) <= n) LOG++;
        up.assign(LOG, vector<int>(n, root));
        depth.assign(n, 0); distRoot.assign(n, 0);
        // iterative DFS to fill parent (up[0]), depth, distRoot
        vector<int> st = {root};
        vector<bool> vis(n, false); vis[root] = true;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (auto [v, w] : adj[u])
                if (!vis[v]) {
```

```
                    vis[v] = true;
                    up[0][v] = u;
                    depth[v] = depth[u] + 1;
                    distRoot[v] = distRoot[u] + w;
                    st.push_back(v);
                }
        }
        // 2^j-th ancestor = 2^(j-1)-th ancestor of the 2^(j-1)-th ancestor
        for (int j = 1; j < LOG; j++)
            for (int v = 0; v < n; v++)
                up[j][v] = up[j - 1][up[j - 1][v]];
    }
    int kth(int v, int k) {                // returns root if k > depth[v]
        for (int j = 0; j < LOG; j++) if (k >> j & 1) v = up[j][v];
        return v;
    }
    int lca(int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        a = kth(a, depth[a] - depth[b]);  // bring a up to b's level
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; j--)  // jump both while ancestors differ
            if (up[j][a] != up[j][b]) { a = up[j][a]; b = up[j][b]; }
        return up[0][a];
    }
    ll dist(int a, int b) { return distRoot[a] + distRoot[b] - 2 *
distRoot[lca(a, b)]; }
    int edgesBetween(int a, int b) { return depth[a] + depth[b] - 2 *
depth[lca(a, b)]; }
};
```

A.14 BRIDGES & ARTICULATION POINTS — UNDIRECTED graph

```
// ============================================================================
// 14. BRIDGES & ARTICULATION POINTS — UNDIRECTED graph
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Bridge = edge whose removal disconnects the graph; AP = such a
vertex.
// COMMON PROBLEM: 'Find all critical connections (network breaks if removed)',
//                 'which routers are single points of failure?', 2-edge-
connected components.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// tin[u] = discovery time, low[u] = smallest tin reachable from u's subtree
// using at most one back edge.
//   Edge (u,v) is a BRIDGE      if low[v] >  tin[u]
//   u is an ARTICULATION POINT  if low[v] >= tin[u] (non-root),
//                               or root with > 1 DFS child.
// Edge ids are used so MULTIPLE EDGES are handled correctly.
// (Recursive: fine for n up to ~2e5; for bigger, increase stack size.)
// ============================================================================
struct BridgesAP {
    int n, m = 0, timer = 0;
    vector<vector<pair<int,int>>> adj;    // (neighbor, edge id)
```

```
    vector<int> tin, low;
    vector<bool> isAP;
    vector<pair<int,int>> bridges;
    BridgesAP(int n) : n(n), adj(n) {}
    void addEdge(int u, int v) {
        adj[u].push_back({v, m});
        adj[v].push_back({u, m});
        m++;
    }
    void dfs(int u, int parentEdge) {
        tin[u] = low[u] = timer++;
        int children = 0;
        for (auto [v, id] : adj[u]) {
            if (id == parentEdge) continue;              // don't go back on
same edge
            if (tin[v] != -1) {                          // back edge
                low[u] = min(low[u], tin[v]);
            } else {                                     // tree edge
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.push_back({u, v});
                if (low[v] >= tin[u] && parentEdge != -1) isAP[u] = true;
                children++;
            }
        }
        if (parentEdge == -1 && children > 1) isAP[u] = true;
    }
    void run() {
        tin.assign(n, -1); low.assign(n, -1); isAP.assign(n, false);
        bridges.clear(); timer = 0;
        for (int i = 0; i < n; i++) if (tin[i] == -1) dfs(i, -1);
    }
};
```

A.15 SCC — Strongly Connected Components (Kosaraju) — DIRECTED graph

```
// ============================================================================
// 15. SCC — Strongly Connected Components (Kosaraju) — DIRECTED graph
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Groups nodes where every node can reach every other (directed
graph).
// COMMON PROBLEM: 'Can every city reach every other? If not, print a failing
pair',
//                 'planets & kingdoms', compress cycles then DP on the DAG (max
coins).
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Step 1: DFS on graph, record nodes by finish time.
// Step 2: DFS on REVERSED graph in decreasing finish time; each tree = 1 SCC.
// comp[v] = SCC id. IDs are in TOPOLOGICAL ORDER of the condensation DAG
// (comp 0 is a "source" component).
// ============================================================================
struct SCC {
```

```
    int n, cnt = 0;
    vector<vector<int>> g, rg;
    vector<int> comp, order;
    vector<bool> vis;
    SCC(int n) : n(n), g(n), rg(n) {}
    void addEdge(int u, int v) { g[u].push_back(v); rg[v].push_back(u); }
    void dfs1(int u) {
        vis[u] = true;
        for (int v : g[u]) if (!vis[v]) dfs1(v);
        order.push_back(u);                      // post-order (finish time)
    }
    void dfs2(int u, int c) {
        comp[u] = c;
        for (int v : rg[u]) if (comp[v] == -1) dfs2(v, c);
    }
    int run() {
        vis.assign(n, false); comp.assign(n, -1); order.clear();
        for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
        reverse(order.begin(), order.end());
        cnt = 0;
        for (int u : order) if (comp[u] == -1) dfs2(u, cnt++);
        return cnt;
    }
    // DAG where each SCC is one node (call after run())
    vector<vector<int>> condensation() {
        vector<vector<int>> dag(cnt);
        for (int u = 0; u < n; u++)
            for (int v : g[u])
                if (comp[u] != comp[v]) dag[comp[u]].push_back(comp[v]);
        for (auto& x : dag) { sort(x.begin(), x.end());
x.erase(unique(x.begin(), x.end()), x.end()); }
        return dag;
    }
};
```

A.16 2-SAT — boolean variables x0..x(n-1), clauses of form (A OR B)

```
// ============================================================================
// 16. 2-SAT — boolean variables x0..x(n-1), clauses of form (A OR B)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Solves boolean formulas where every clause has exactly 2
literals.
// COMMON PROBLEM: 'Each person wants topping A or not B — satisfy everyone?',
//                 'pick one of 2 options per item with pairwise conflicts'.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Literal "x = true" is node 2x, "x = false" is node 2x+1.
// Clause (A or B) gives implications: (!A -> B) and (!B -> A).
// Unsatisfiable iff x and !x are in the same SCC.
// Assignment: x = true if comp[x_true] comes AFTER comp[x_false] in topo order.
// ============================================================================
struct TwoSAT {
    int n;
```

```
    SCC scc;
    vector<bool> value;
    TwoSAT(int n) : n(n), scc(2 * n) {}
    int lit(int x, bool val) { return 2 * x + (val ? 0 : 1); }
    // (x == vx) OR (y == vy)
    void addOr(int x, bool vx, int y, bool vy) {
        int a = lit(x, vx), b = lit(y, vy);
        scc.addEdge(a ^ 1, b);    // !a -> b
        scc.addEdge(b ^ 1, a);    // !b -> a
    }
    void forceValue(int x, bool vx) { addOr(x, vx, x, vx); }              // x
must be vx
    void addImplies(int x, bool vx, int y, bool vy) { addOr(x, !vx, y, vy); } //
(x==vx) -> (y==vy)
    void addXor(int x, int y) { addOr(x, true, y, true); addOr(x, false, y,
false); } // exactly one true
    void addAtMostOne(int x, int y) { addOr(x, false, y, false); }         //
not both true
    bool solve() {
        scc.run();
        value.assign(n, false);
        for (int i = 0; i < n; i++) {
            if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;
            value[i] = scc.comp[2 * i] > scc.comp[2 * i + 1];
        }
        return true;
    }
};
```

A.17 MAX FLOW — DINIC

```
// ============================================================================
// 17. MAX FLOW — DINIC
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Maximum amount of 'stuff' you can push from source s to sink
t.
// COMMON PROBLEM: 'Max download speed from server to computer', 'max edge-
disjoint paths',
//                 min cut: 'cheapest edges to remove to separate s and t',
project selection.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// addEdge(u, v, cap) adds a directed edge. For an UNDIRECTED edge use
// addEdge(u, v, cap, cap).
// Edges are stored in pairs: edge id and id^1 is its reverse edge.
// After maxflow:
//   - flow on edge id = (original cap) - e[id].cap
//   - minCut(): nodes reachable from s in the residual graph (S-side).
//     Edges going from S-side to T-side form the minimum cut.
// ============================================================================
struct Dinic {
    struct E { int to; ll cap; };
    int n;
    vector<E> e;
```

```
    vector<vector<int>> g;
    vector<int> level, it;
    Dinic(int n) : n(n), g(n) {}
    int addEdge(int u, int v, ll cap, ll revCap = 0) {
        g[u].push_back(e.size()); e.push_back({v, cap});
        g[v].push_back(e.size()); e.push_back({u, revCap});
        return e.size() - 2;                        // id of forward edge
    }
    bool bfs(int s, int t) {                        // build level graph
        level.assign(n, -1);
        queue<int> q; level[s] = 0; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : g[u])
                if (e[id].cap > 0 && level[e[id].to] == -1) {
                    level[e[id].to] = level[u] + 1;
                    q.push(e[id].to);
                }
        }
        return level[t] != -1;
    }
    ll dfs(int u, int t, ll f) {                    // send blocking flow
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i];
            int v = e[id].to;
            if (e[id].cap > 0 && level[v] == level[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) {
                    e[id].cap -= d;
                    e[id ^ 1].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) {
            it.assign(n, 0);
            while (ll f = dfs(s, t, INF)) flow += f;
        }
        return flow;
    }
    vector<bool> minCut(int s) {                    // call after maxflow
        bfs(s, s);
        vector<bool> side(n);
        for (int i = 0; i < n; i++) side[i] = level[i] != -1;
        return side;
    }
};
```

A.18 MIN COST MAX FLOW — SPFA (Bellman-Ford with queue)

```
// ================================================================================
// 18. MIN COST MAX FLOW — SPFA (Bellman-Ford with queue)
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Max flow, but picks the cheapest one among all max flows.
// COMMON PROBLEM: 'Ship k units of goods with minimum transport cost',
//                 'assign tasks to workers with costs and capacities'.
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Sends as much flow as possible (up to maxF) with minimum total cost.
// Works with negative costs as long as there are no negative cycles.
// run(s, t) returns {flow, cost}.
// ================================================================================
struct MCMF {
    struct E { int to; ll cap, cost; };
    int n;
    vector<E> e;
    vector<vector<int>> g;
    vector<ll> dist;
    vector<int> parEdge;
    vector<bool> inq;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back(e.size()); e.push_back({v, cap, cost});
        g[v].push_back(e.size()); e.push_back({u, 0, -cost});
    }
    bool spfa(int s, int t) {                    // cheapest augmenting path
        dist.assign(n, INF); parEdge.assign(n, -1); inq.assign(n, false);
        queue<int> q; dist[s] = 0; q.push(s); inq[s] = true;
        while (!q.empty()) {
            int u = q.front(); q.pop(); inq[u] = false;
            for (int id : g[u]) {
                auto& ed = e[id];
                if (ed.cap > 0 && dist[u] + ed.cost < dist[ed.to]) {
                    dist[ed.to] = dist[u] + ed.cost;
                    parEdge[ed.to] = id;
                    if (!inq[ed.to]) { inq[ed.to] = true; q.push(ed.to); }
                }
            }
        }
        return dist[t] < INF;
    }
    pair<ll,ll> run(int s, int t, ll maxF = INF) {
        ll flow = 0, cost = 0;
        while (flow < maxF && spfa(s, t)) {
            ll f = maxF - flow;                  // bottleneck on the path
            for (int v = t; v != s; v = e[parEdge[v] ^ 1].to) f = min(f,
e[parEdge[v]].cap);
            for (int v = t; v != s; v = e[parEdge[v] ^ 1].to) {
                e[parEdge[v]].cap -= f;
                e[parEdge[v] ^ 1].cap += f;
            }
```

```
            flow += f; cost += f * dist[t];
        }
        return {flow, cost};
    }
};
```

A.19 BIPARTITE MATCHING — HOPCROFT-KARP

```
// ================================================================================
// 19. BIPARTITE MATCHING — HOPCROFT-KARP
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Maximum number of pairs between left and right, no node used
twice.
// COMMON PROBLEM: 'Max boy-girl dance pairs', 'assign jobs to qualified
workers',
//                 min vertex cover on bipartite graph, min path cover in DAG (n
- matching).
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Left side: 0..nL-1, right side: 0..nR-1. addEdge(l, r).
// matchL[l] = right partner (-1 if none), matchR[r] = left partner.
// Useful facts (Konig): in bipartite graphs
//   min vertex cover = max matching
//   max independent set = V - max matching
//   min path cover in DAG = n - max matching (split each node into in/out)
// ================================================================================
struct HopcroftKarp {
    int nL, nR;
    vector<vector<int>> g;
    vector<int> matchL, matchR, dist;
    HopcroftKarp(int nL, int nR) : nL(nL), nR(nR), g(nL) {}
    void addEdge(int l, int r) { g[l].push_back(r); }
    bool bfs() {                          // layer free left nodes
        queue<int> q;
        dist.assign(nL, -1);
        for (int l = 0; l < nL; l++) if (matchL[l] == -1) { dist[l] = 0;
q.push(l); }
        bool found = false;
        while (!q.empty()) {
            int l = q.front(); q.pop();
            for (int r : g[l]) {
                int l2 = matchR[r];
                if (l2 == -1) found = true;           // free right node
reachable
                else if (dist[l2] == -1) { dist[l2] = dist[l] + 1; q.push(l2); }
            }
        }
        return found;
    }
    bool dfs(int l) {                     // find augmenting path
        for (int r : g[l]) {
            int l2 = matchR[r];
            if (l2 == -1 || (dist[l2] == dist[l] + 1 && dfs(l2))) {
                matchL[l] = r; matchR[r] = l;
```

```
                return true;
            }
        }
        dist[l] = -1;
        return false;
    }
    int maxMatching() {
        matchL.assign(nL, -1); matchR.assign(nR, -1);
        int res = 0;
        while (bfs())
            for (int l = 0; l < nL; l++)
                if (matchL[l] == -1 && dfs(l)) res++;
        return res;
    }
};
```

A.20 HUNGARIAN ALGORITHM — minimum cost assignment

```
// ================================================================================
// 20. HUNGARIAN ALGORITHM — minimum cost assignment
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Perfect assignment of n workers to m jobs minimizing total
cost.
// COMMON PROBLEM: 'n workers, n jobs, cost matrix: minimum total cost',
//                 'maximum total profit assignment' (negate costs), n up to
~500.
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// cost[i][j] = cost of assigning worker i to job j (n workers, m jobs, n <= m).
// Returns {min total cost, assign} where assign[i] = job of worker i.
// For MAXIMUM: negate all costs, then negate the answer.
// ================================================================================
pair<ll, vector<int>> hungarian(const vector<vector<ll>>& cost) {
    int n = cost.size(), m = cost[0].size();
    vector<ll> u(n + 1, 0), v(m + 1, 0);           // potentials
    vector<int> p(m + 1, 0), way(m + 1, 0);        // p[j] = row matched to
column j
    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<ll> minv(m + 1, INF);
        vector<bool> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            for (int j = 1; j <= m; j++)
                if (!used[j]) {
                    ll cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
                    if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
                    if (minv[j] < delta) { delta = minv[j]; j1 = j; }
                }
            for (int j = 0; j <= m; j++)
                if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
```

```
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do { int j1 = way[j0]; p[j0] = p[j1]; j0 = j1; } while (j0);
    }
    vector<int> assign(n, -1);
    for (int j = 1; j <= m; j++) if (p[j]) assign[p[j] - 1] = j - 1;
    return {-v[0], assign};
}
```

A.21 EULER PATH / CIRCUIT — HIERHOLZER (uses every EDGE exactly once)

```
// ============================================================================
// 21. EULER PATH / CIRCUIT — HIERHOLZER (uses every EDGE exactly once)
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Walks through every EDGE exactly once (path or closed
circuit).
// COMMON PROBLEM: 'Mail delivery: use every street once and return home',
//                 'rebuild string from all k-length substrings' (De Bruijn),
'teleporters path'.
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Conditions:
//   Undirected: 0 odd-degree nodes (circuit) or exactly 2 (path).
//   Directed  : all in == out (circuit) or one node out-in=1 (start)
//               and one node in-out=1 (end).
//   All edges must be in one connected component.
// find() returns the sequence of vertices (m+1 of them), or {} if impossible.
// ============================================================================
struct Euler {
    int n, m = 0;
    bool directed;
    vector<vector<pair<int,int>>> g;   // (to, edge id)
    vector<int> indeg, outdeg;
    Euler(int n, bool directed) : n(n), directed(directed), g(n), indeg(n, 0),
outdeg(n, 0) {}
    void addEdge(int u, int v) {
        g[u].push_back({v, m});
        if (!directed) g[v].push_back({u, m});
        outdeg[u]++; indeg[v]++;
        m++;
    }
    vector<int> find() {
        int start = -1;
        if (directed) {
            int plus = 0, minus = 0;
            for (int v = 0; v < n; v++) {
                int d = outdeg[v] - indeg[v];
                if (d == 1) { plus++; start = v; }
                else if (d == -1) minus++;
                else if (d != 0) return {};
            }
            if (plus > 1 || plus != minus) return {};
```

```
        } else {
            int odd = 0;
            for (int v = 0; v < n; v++)
                if ((indeg[v] + outdeg[v]) % 2) { odd++; start = v; }
            if (odd != 0 && odd != 2) return {};
        }
        if (start == -1)                        // circuit: start anywhere with
an edge
            for (int v = 0; v < n; v++) if (!g[v].empty()) { start = v; break; }
        if (start == -1) return {};             // no edges at all

        vector<int> ptr(n, 0), path, st = {start};
        vector<bool> used(m, false);
        while (!st.empty()) {
            int u = st.back();
            while (ptr[u] < (int)g[u].size() && used[g[u][ptr[u]].second])
ptr[u]++;
            if (ptr[u] == (int)g[u].size()) {  // dead end -> add to answer
                path.push_back(u); st.pop_back();
            } else {
                auto [v, id] = g[u][ptr[u]];
                used[id] = true;
                st.push_back(v);
            }
        }
        if ((int)path.size() != m + 1) return {};   // graph not connected
        reverse(path.begin(), path.end());
        return path;
    }
};
```

A.22 TREE DIAMETER — longest path in a tree

```
// ============================================================================
// 22. TREE DIAMETER — longest path in a tree
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Length of the longest path between any two nodes in a tree.
// COMMON PROBLEM: 'Longest cable between two computers in a tree network',
//                 'for each node, farthest node distance' = max(dist from A,
dist from B).
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Trick: from any node, the farthest node A is one end of the diameter.
// The farthest node from A is the other end B.
// ============================================================================
vector<ll> treeDistances(int src, const vector<vector<pair<int,ll>>>& adj) {
    int n = adj.size();
    vector<ll> d(n, -1);
    vector<int> st = {src};
    d[src] = 0;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        for (auto [v, w] : adj[u]) if (d[v] == -1) { d[v] = d[u] + w;
st.push_back(v); }
```

```
    }
    return d;
}

// returns {diameter length, endpoint A, endpoint B}
tuple<ll,int,int> treeDiameter(const vector<vector<pair<int,ll>>>& adj) {
    auto d0 = treeDistances(0, adj);
    int A = max_element(d0.begin(), d0.end()) - d0.begin();
    auto dA = treeDistances(A, adj);
    int B = max_element(dA.begin(), dA.end()) - dA.begin();
    return {dA[B], A, B};
}
```

A.23 HEAVY-LIGHT DECOMPOSITION — path queries/updates on a tree

```
// ============================================================================
// 23. HEAVY-LIGHT DECOMPOSITION — path queries/updates on a tree
// ----------------------------------------------------------------------------
// WHAT IT DOES  : Breaks a tree into chains so path queries become O(log)
segment-tree ranges.
// COMMON PROBLEM: 'Change value of node v; query sum/max on path a->b',
//                 'path queries with point updates' (CSES Path Queries II).
// ----------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Splits the tree into chains so any path crosses O(log n) chains.
// Each chain occupies a contiguous range in the segment tree.
// Here: values on NODES, point update, path SUM query.
// (Change SegTree's operation to max/min/etc. as needed.
//  For values on EDGES: store edge (par[v], v) at node v, and skip the
//  LCA node in the last query: st.query(pos[a] + 1, pos[b]).)
// ============================================================================
struct SegTree {                        // iterative segment tree, sum
    int n;
    vector<ll> t;
    SegTree(int n) : n(n), t(2 * n, 0) {}
    void update(int p, ll val) {        // set position p = val
        for (t[p += n] = val; p > 1; p >>= 1) t[p >> 1] = t[p] + t[p ^ 1];
    }
    ll query(int l, int r) {            // sum on [l, r] inclusive
        ll res = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += t[l++];
            if (r & 1) res += t[--r];
        }
        return res;
    }
};

struct HLD {
    int n, cur = 0;
    vector<int> par, dep, heavy, head, pos, sz;
    SegTree st;
    HLD(const vector<vector<int>>& adj, int root = 0)
```

```
        : n(adj.size()), par(n, -1), dep(n, 0), heavy(n, -1),
          head(n), pos(n), sz(n, 1), st(n) {
        // BFS order (no recursion)
        vector<int> order = {root};
        for (int i = 0; i < (int)order.size(); i++) {
            int u = order[i];
            for (int v : adj[u]) if (v != par[u]) { par[v] = u; dep[v] = dep[u]
+ 1; order.push_back(v); }
        }
        // subtree sizes, bottom-up
        for (int i = (int)order.size() - 1; i > 0; i--) sz[par[order[i]]] +=
sz[order[i]];
        // heavy child = child with the biggest subtree
        for (int u : order)
            for (int v : adj[u])
                if (v != par[u] && (heavy[u] == -1 || sz[v] > sz[heavy[u]]))
heavy[u] = v;
        // walk every chain from its top, giving consecutive positions
        for (int u : order)
            if (par[u] == -1 || heavy[par[u]] != u)
                for (int v = u; v != -1; v = heavy[v]) { head[v] = u; pos[v] =
cur++; }
    }
    void update(int v, ll val) { st.update(pos[v], val); }
    ll queryPath(int a, int b) {
        ll res = 0;
        // climb from the deeper chain head until both are on the same chain
        for (; head[a] != head[b]; b = par[head[b]]) {
            if (dep[head[a]] > dep[head[b]]) swap(a, b);
            res += st.query(pos[head[b]], pos[b]);
        }
        if (dep[a] > dep[b]) swap(a, b);
        res += st.query(pos[a], pos[b]);
        return res;
    }
    int lca(int a, int b) {
        for (; head[a] != head[b]; b = par[head[b]])
            if (dep[head[a]] > dep[head[b]]) swap(a, b);
        return dep[a] < dep[b] ? a : b;
    }
};
```

A.24 CENTROID DECOMPOSITION (skeleton)

```
// ================================================================================
// 24. CENTROID DECOMPOSITION (skeleton)
// --------------------------------------------------------------------------------
// WHAT IT DOES  : Recursively splits a tree at centroids; every path goes
through some centroid.
// COMMON PROBLEM: 'Count pairs of nodes at distance exactly k',
//                 'nearest red node with paint updates' (Xenia and Tree).
// --------------------------------------------------------------------------------
// HOW IT WORKS / NOTES:
// Centroid = node whose removal leaves pieces of size <= n/2.
```

```cpp
// Recursively removing centroids gives a "centroid tree" of depth O(log n).
// Every path in the original tree passes through the centroid of some
// level, so: "count/handle all paths through c" at each centroid c.
// cpar[v] = parent of v in the centroid tree (-1 for the top).
// Put your problem logic where it says  >>> PROCESS <<<.
// ================================================================================
struct CentroidDecomp {
    vector<vector<int>> adj;
    vector<int> sz, cpar;
    vector<bool> removed;
    CentroidDecomp(const vector<vector<int>>& a)
        : adj(a), sz(a.size()), cpar(a.size(), -1), removed(a.size(), false) {
        build(0, -1);
    }
    int calcSize(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) if (v != p && !removed[v]) sz[u] += calcSize(v, u);
        return sz[u];
    }
    int findCentroid(int u, int p, int total) {
        for (int v : adj[u])
            if (v != p && !removed[v] && sz[v] * 2 > total) return
findCentroid(v, u, total);
        return u;
    }
    void build(int u, int p) {
        int total = calcSize(u, -1);
        int c = findCentroid(u, -1, total);
        cpar[c] = p;
        removed[c] = true;
        // >>> PROCESS all paths passing through centroid c here <<<
        for (int v : adj[c]) if (!removed[v]) build(v, c);
    }
};
```

A.25 Demo main (shows how to call every algorithm)

```cpp
// ================================================================================
//                                DEMO / TESTS
// ================================================================================
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // ---------- Unweighted undirected graph ----------
    //  0 - 1 - 2 - 3      4 - 5
    int n = 6;
    vector<vector<int>> adj(n);
    auto addU = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
    addU(0, 1); addU(1, 2); addU(2, 3); addU(4, 5);

    BFS bfs; bfs.run(0, adj);
    cout << "[1] BFS dist 0->3 = " << bfs.dist[3] << ", path:";
    for (int v : bfs.path(3)) cout << ' ' << v;
```

```
    cout << "\n";

    vector<string> grid = {"..#", ".##", "..."};
    cout << "[2] Grid BFS (0,0)->(2,2) = " << gridBFS(grid, 0, 0)[2][2] << "\n";

    cout << "[3] Components = " << countComponents(adj) << "\n";

    vector<vector<pair<int,int>>> g01(3);
    g01[0].push_back({1, 1}); g01[0].push_back({2, 1}); g01[2].push_back({1,
0});
    cout << "[4] 0-1 BFS dist 0->1 = " << zeroOneBFS(0, g01)[1] << "\n";

    DSU dsu(5); dsu.unite(0, 1); dsu.unite(1, 2);
    cout << "[5] DSU same(0,2)=" << dsu.same(0, 2) << " size(0)=" << dsu.size(0)
<< "\n";

    // ---------- Directed graph for topo sort ----------
    vector<vector<int>> dag(4);
    dag[0] = {1, 2}; dag[1] = {3}; dag[2] = {3};
    cout << "[6] Topo order:";
    for (int v : topoSort(dag)) cout << ' ' << v;
    cout << "\n";

    vector<int> color;
    cout << "[7] Bipartite = " << isBipartite(adj, color) << "\n";

    // ---------- Weighted graph ----------
    vector<vector<pair<int,ll>>> wadj(4);
    auto addW = [&](int u, int v, ll w) { wadj[u].push_back({v, w});
wadj[v].push_back({u, w}); };
    addW(0, 1, 4); addW(0, 2, 1); addW(2, 1, 2); addW(1, 3, 5);
    Dijkstra dj; dj.run(0, wadj);
    cout << "[8] Dijkstra 0->3 = " << dj.dist[3] << ", path:";
    for (int v : dj.path(3)) cout << ' ' << v;
    cout << "\n";

    vector<Edge> edges = {{0, 1, 4}, {0, 2, 1}, {2, 1, -2}, {1, 3, 5}};
    vector<ll> bf;
    bool ok = bellmanFord(4, 0, edges, bf);
    cout << "[9] Bellman-Ford ok=" << ok << " dist 0->3 = " << bf[3] << "\n";

    vector<vector<ll>> fw(4, vector<ll>(4, INF));
    for (int i = 0; i < 4; i++) fw[i][i] = 0;
    for (auto& e : edges) fw[e.u][e.v] = min(fw[e.u][e.v], e.w);
    floydWarshall(fw);
    cout << "[10] Floyd 0->3 = " << fw[0][3] << "\n";

    vector<Edge> und = {{0, 1, 4}, {0, 2, 1}, {2, 1, 2}, {1, 3, 5}};
    cout << "[11] Kruskal MST = " << kruskal(4, und) << "\n";
    cout << "[12] Prim MST = " << prim(wadj) << "\n";

    // ---------- Tree ----------
    //        0
    //       / \    (tree)
    //      1   2
    //     / \   \  (tree)
```

```
    //    3   4   5
    vector<vector<pair<int,ll>>> tree(6);
    vector<vector<int>> tadj(6);
    auto addT = [&](int u, int v) {
        tree[u].push_back({v, 1}); tree[v].push_back({u, 1});
        tadj[u].push_back(v); tadj[v].push_back(u);
    };
    addT(0, 1); addT(0, 2); addT(1, 3); addT(1, 4); addT(2, 5);
    LCA lca(tree, 0);
    cout << "[13] LCA(3,4)=" << lca.lca(3, 4) << " LCA(3,5)=" << lca.lca(3, 5)
         << " dist(3,5)=" << lca.dist(3, 5) << "\n";

    // ---------- Bridges & APs ----------
    //  0 - 1 - 2      (1-2-3 is a triangle; 0-1 is a bridge)
    //       \ /
    //        3
    BridgesAP bap(4);
    bap.addEdge(0, 1); bap.addEdge(1, 2); bap.addEdge(2, 3); bap.addEdge(3, 1);
    bap.run();
    cout << "[14] Bridges:";
    for (auto [u, v] : bap.bridges) cout << " (" << u << "," << v << ")";
    cout << "  APs:";
    for (int i = 0; i < 4; i++) if (bap.isAP[i]) cout << ' ' << i;
    cout << "\n";

    // ---------- SCC ----------
    SCC scc(5);
    scc.addEdge(0, 1); scc.addEdge(1, 2); scc.addEdge(2, 0); scc.addEdge(2, 3);
scc.addEdge(3, 4);
    cout << "[15] SCC count = " << scc.run() << "\n";

    // ---------- 2-SAT ----------
    // (x0 OR x1) AND (NOT x0 OR x1) AND (NOT x1 OR NOT x2)  => x1 = true, x2 =
false
    TwoSAT ts(3);
    ts.addOr(0, true, 1, true);
    ts.addOr(0, false, 1, true);
    ts.addOr(1, false, 2, false);
    bool sat = ts.solve();
    cout << "[16] 2-SAT sat=" << sat << " values:";
    for (int i = 0; i < 3; i++) cout << ' ' << ts.value[i];
    cout << "\n";

    // ---------- Max flow ----------
    Dinic din(4);
    din.addEdge(0, 1, 3); din.addEdge(0, 2, 2); din.addEdge(1, 2, 5);
    din.addEdge(1, 3, 2); din.addEdge(2, 3, 3);
    cout << "[17] Max flow = " << din.maxflow(0, 3) << "\n";

    // ---------- Min cost max flow ----------
    MCMF mc(4);
    mc.addEdge(0, 1, 2, 1); mc.addEdge(0, 2, 1, 2);
    mc.addEdge(1, 3, 1, 3); mc.addEdge(1, 2, 1, 1); mc.addEdge(2, 3, 2, 1);
    auto [fl, co] = mc.run(0, 3);
    cout << "[18] MCMF flow=" << fl << " cost=" << co << "\n";
```

```
    // ---------- Bipartite matching ----------
    HopcroftKarp hk(3, 3);
    hk.addEdge(0, 0); hk.addEdge(0, 1); hk.addEdge(1, 0); hk.addEdge(2, 2);
    cout << "[19] Max matching = " << hk.maxMatching() << "\n";

    // ---------- Hungarian ----------
    vector<vector<ll>> cst = {{4, 1, 3}, {2, 0, 5}, {3, 2, 2}};
    auto [hc, asg] = hungarian(cst);
    cout << "[20] Hungarian cost=" << hc << " assign:";
    for (int j : asg) cout << ' ' << j;
    cout << "\n";

    // ---------- Euler ----------
    Euler eu(4, false);
    eu.addEdge(0, 1); eu.addEdge(1, 2); eu.addEdge(2, 0); eu.addEdge(0, 3);
    cout << "[21] Euler path:";
    for (int v : eu.find()) cout << ' ' << v;
    cout << "\n";

    auto [diam, A, B] = treeDiameter(tree);
    cout << "[22] Tree diameter = " << diam << " (" << A << " .. " << B <<
")\n";

    // ---------- HLD ----------
    HLD hld(tadj, 0);
    for (int v = 0; v < 6; v++) hld.update(v, v + 1);   // value of node v = v+1
    cout << "[23] HLD path sum 3->5 = " << hld.queryPath(3, 5)
         << "  lca(4,3)=" << hld.lca(4, 3) << "\n";

    CentroidDecomp cd(tadj);
    cout << "[24] Centroid tree parent:";
    for (int v = 0; v < 6; v++) cout << ' ' << cd.cpar[v];
    cout << "\n";
    return 0;
}
```

Appendix B. Bitmask DP Template (Elevator Rides worked solution)

```
/*
 ================================================================================
   BITMASK DP TEMPLATE — learned by solving one problem
 ================================================================================

   PROBLEM: Elevator Rides  (CSES 1653, https://cses.fi/problemset/task/1653)
   ------------------------------------------------------------------------------
   There are n people who want to reach the top floor of a building.
   The elevator can carry at most x total weight in one ride.
   Person i has weight w[i].
   What is the MINIMUM number of rides needed to take everyone up?

   Input
     First line : n x          (1 <= n <= 20, 1 <= x <= 1e9)
     Second line: w1 w2 ... wn (1 <= wi <= x)
   Output
     One integer: the minimum number of rides.

   Example
     Input:
       4 10
       4 8 6 1
     Output:
       2
     (Ride 1: 4 + 6 = 10, Ride 2: 8 + 1 = 9)

 ================================================================================
   SOLUTION IDEA
 ================================================================================
   Why bitmask?  n <= 20 is the BIG HINT. 2^20 ~ 1e6 subsets, so we can
   afford a DP over every subset of people.

   A "mask" is an integer whose bits say which people are already up:
       bit i = 1  ->  person i has already ridden
       mask = 0b0101 -> persons 0 and 2 are up

   STATE
     dp[mask] = { rides, last }
       rides = minimum number of rides used to bring up exactly the set "mask"
       last  = weight in the CURRENT (last) ride, as small as possible
               (a lighter last ride leaves more room for the next person)

   Comparing two states: fewer rides is better; if equal, smaller "last".
   std::pair compares exactly like that, so we just use min().

   TRANSITION  (add person i who is NOT in mask yet)
     if last + w[i] <= x   ->  same ride:  {rides, last + w[i]}
     else                  ->  new ride :  {rides + 1, w[i]}
     dp[mask | (1<<i)] = min(dp[mask | (1<<i)], that)
```

```
   BASE
     dp[0] = {1, 0}   // "1 ride started, 0 weight inside" (avoids a special
case)

   ANSWER
     dp[(1<<n) - 1].first     // all bits set = everybody is up

   ORDER
     mask | (1<<i) is always BIGGER than mask, so a simple loop
     mask = 0 .. 2^n - 1 processes every state before it is used.

   COMPLEXITY: O(2^n * n) time, O(2^n) memory  ->  ~2e7 ops for n = 20.
 ================================================================================
*/
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// ================================================================================
// PART 1: BIT OPERATIONS CHEAT SHEET (use these in any bitmask problem)
// ================================================================================
//   (mask >> i) & 1        -> is bit i set?  (is item i in the set?)
//   mask | (1 << i)        -> add item i
//   mask & ~(1 << i)       -> remove item i
//   mask ^ (1 << i)        -> toggle item i
//   (1 << n) - 1           -> full set {0, 1, ..., n-1}
//   __builtin_popcount(m)  -> number of items in the set  (popcountll for long
long)
//   __builtin_ctz(m)       -> index of the lowest set bit (m != 0)
//   m & -m                 -> value of the lowest set bit
//   m & (m - 1)            -> remove the lowest set bit
//   (a & b) == a           -> a is a subset of b
//   Use 1LL << i when i can be >= 31.

// ================================================================================
// PART 2: SOLUTION OF THE PROBLEM
// ================================================================================
int solveElevator(int n, ll x, const vector<ll>& w) {
    int FULL = (1 << n) - 1;
    // dp[mask] = {rides, weight of the last ride}
    vector<pair<int, ll>> dp(1 << n, {INT_MAX, 0});
    dp[0] = {1, 0};                                  // base case

    for (int mask = 0; mask <= FULL; mask++) {
        if (dp[mask].first == INT_MAX) continue;  // unreachable state (never
happens here)
        auto [rides, last] = dp[mask];
        for (int i = 0; i < n; i++) {
            if (mask >> i & 1) continue;          // person i is already up
            pair<int, ll> nxt;
            if (last + w[i] <= x) nxt = {rides, last + w[i]};   // fits in
current ride
            else                  nxt = {rides + 1, w[i]};      // start a new
ride
            int nmask = mask | (1 << i);
```

```
            dp[nmask] = min(dp[nmask], nxt);
        }
    }
    return dp[FULL].first;
}

// ============================================================================
// PART 3: OTHER BITMASK DP PATTERNS (reusable templates)
// ============================================================================

// ---- 3a. Travelling Salesman / Hamiltonian path ----------------------------
// dp[mask][v] = min cost of a path that visits exactly the nodes in mask
//               and ends at v.  Start at node 0, return to 0.
// Same shape counts Hamiltonian paths (CSES 1690 Hamiltonian Flights):
// replace min with += (mod 1e9+7).
// Complexity: O(2^n * n^2), n <= ~20.
ll tsp(int n, const vector<vector<ll>>& cost) {
    const ll INF = 1e18;
    vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
    dp[1][0] = 0;                                   // only node 0 visited,
standing at 0
    for (int mask = 1; mask < (1 << n); mask++)
        for (int v = 0; v < n; v++) {
            if (dp[mask][v] == INF || !(mask >> v & 1)) continue;
            for (int u = 0; u < n; u++) {
                if (mask >> u & 1) continue;        // already visited
                int nm = mask | (1 << u);
                dp[nm][u] = min(dp[nm][u], dp[mask][v] + cost[v][u]);
            }
        }
    ll best = INF;
    for (int v = 1; v < n; v++)                     // close the cycle back to 0
        best = min(best, dp[(1 << n) - 1][v] + cost[v][0]);
    return n == 1 ? 0 : best;
}

// ---- 3b. Iterate over all SUBMASKS of a mask -------------------------------
// Total over all masks = O(3^n)  (fine for n <= ~15).
// Typical use: split a set into groups, dp[mask] = min over sub of dp[mask^sub]
+ cost[sub].
void submaskExample(int mask) {
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
        // sub is a non-empty submask of mask
        // the rest is mask ^ sub
    }
}

// ---- 3c. Sum over subsets (SOS DP) -----------------------------------------
// f[mask] = sum of a[sub] for every sub that is a subset of mask.  O(2^n * n).
// (CSES 1654 SOS Bit Problem uses this.)
vector<ll> sosDP(int n, vector<ll> f) {
    for (int i = 0; i < n; i++)
        for (int mask = 0; mask < (1 << n); mask++)
            if (mask >> i & 1) f[mask] += f[mask ^ (1 << i)];
    return f;
}
```

```cpp
// ---- 3d. Brute force over all subsets -------------------------------------
// Check every subset of n <= 20 items (e.g. "pick items with total exactly S").
bool subsetSumExists(const vector<ll>& a, ll S) {
    int n = a.size();
    for (int mask = 0; mask < (1 << n); mask++) {
        ll sum = 0;
        for (int i = 0; i < n; i++) if (mask >> i & 1) sum += a[i];
        if (sum == S) return true;
    }
    return false;
}

// ================================================================================
// MAIN: reads the CSES input and prints the answer.
// Run with no input file to see the demos:  ./a < /dev/null
// ================================================================================
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n; ll x;
    if (cin >> n >> x) {                          // CSES judge input
        vector<ll> w(n);
        for (auto& v : w) cin >> v;
        cout << solveElevator(n, x, w) << "\n";
        return 0;
    }

    // ---- demos (when no input is given) ----
    cout << "Elevator example: " << solveElevator(4, 10, {4, 8, 6, 1}) << "
(expected 2)\n";
    vector<vector<ll>> c = {{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30},
{20, 25, 30, 0}};
    cout << "TSP example: " << tsp(4, c) << " (expected 80)\n";
    auto f = sosDP(2, {1, 2, 3, 4});
    cout << "SOS f[3] = " << f[3] << " (expected 10)\n";
    cout << "Subset sum 9 in {3,34,4,12,5,2}: " << subsetSumExists({3, 34, 4,
12, 5, 2}, 9) << " (expected 1)\n";
    return 0;
}
```